AWS VPC Peering and EC2 Connectivity

Experiment: VPC Creation and Internet Gateway Configuration

Date: October 23, 2026

1 Aim

To create two Virtual Private Clouds (VPCs) in different AWS regions, Mumbai and Ohio, configure their IPv4 CIDR blocks, create public subnets, and attach Internet Gateways. These VPCs will subsequently be connected using VPC Peering to enable private communication between EC2 instances.

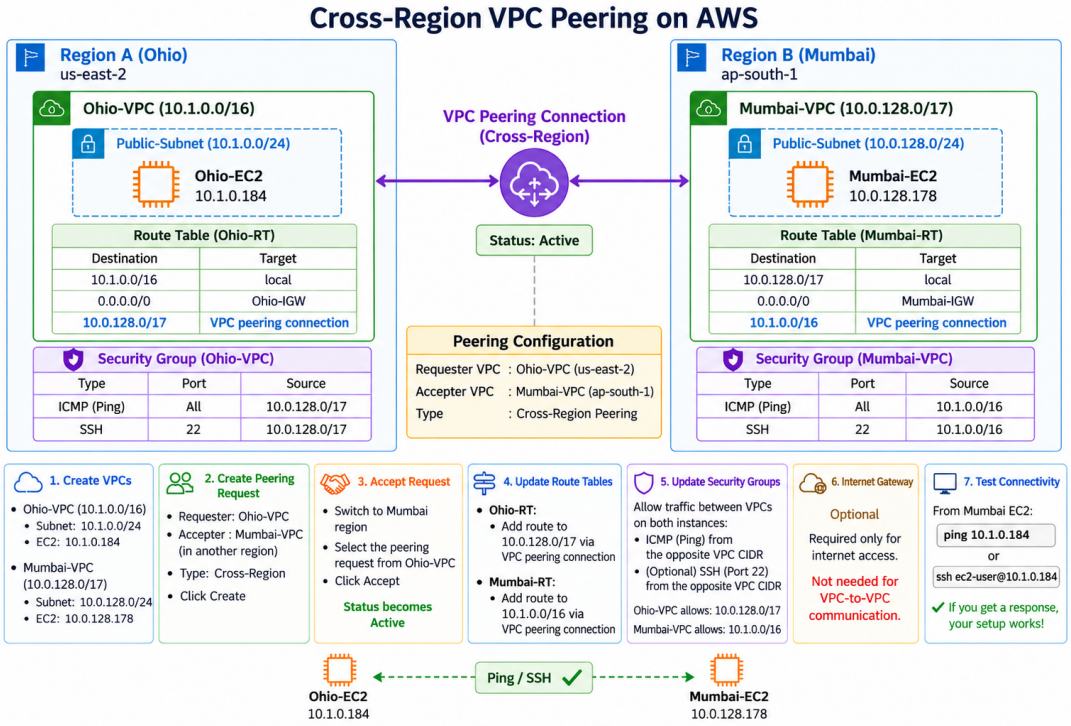


Figure 1: Topology of the Ohio and Mumbai VPCs

2 Objectives

1. Create a VPC in the US East (Ohio) region.
2. Configure its IPv4 CIDR block as `10.1.0.0/16`.
3. Create a subnet with CIDR `10.1.0.0/24`.
4. Create and attach an Internet Gateway to Ohio-VPC.
5. Create a VPC in the Asia Pacific (Mumbai) region.
6. Configure its IPv4 CIDR block as `10.0.128.0/17`.

7. Create a subnet with CIDR `10.0.128.0/24`.

8. Create and attach an Internet Gateway to Mumbai-VPC.

3 Resources Used

- AWS Management Console
- Amazon Virtual Private Cloud (VPC)
- IPv4 CIDR addressing
- Subnets
- Internet Gateways (IGW)
- AWS Regions: Ohio (`us-east-2`) and Mumbai (`ap-south-1`)

4 Procedure

4.1 Step 1: Open the VPC Dashboard

Sign in to the AWS Management Console and open the Amazon VPC service. Select the US East (Ohio) region (`us-east-2`) from the region selector.

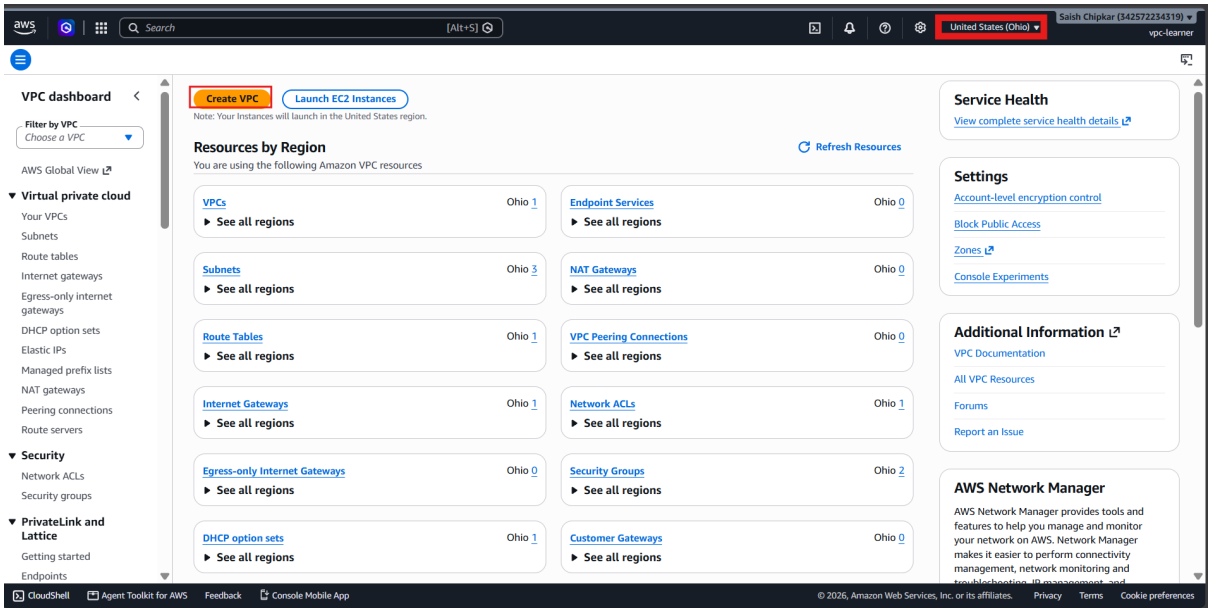


Figure 2: VPC dashboard in the Ohio region

4.2 Step 2: Create the Ohio VPC

Click **Create VPC** and select **VPC only** under Resources to create.
Enter the following configuration:

- Name tag: `Ohio-VPC`
- IPv4 CIDR block: `10.1.0.0/16`
- IPv6 CIDR block: No IPv6 CIDR block
- Tenancy: Default
- VPC encryption control: None

Click **Create VPC**.

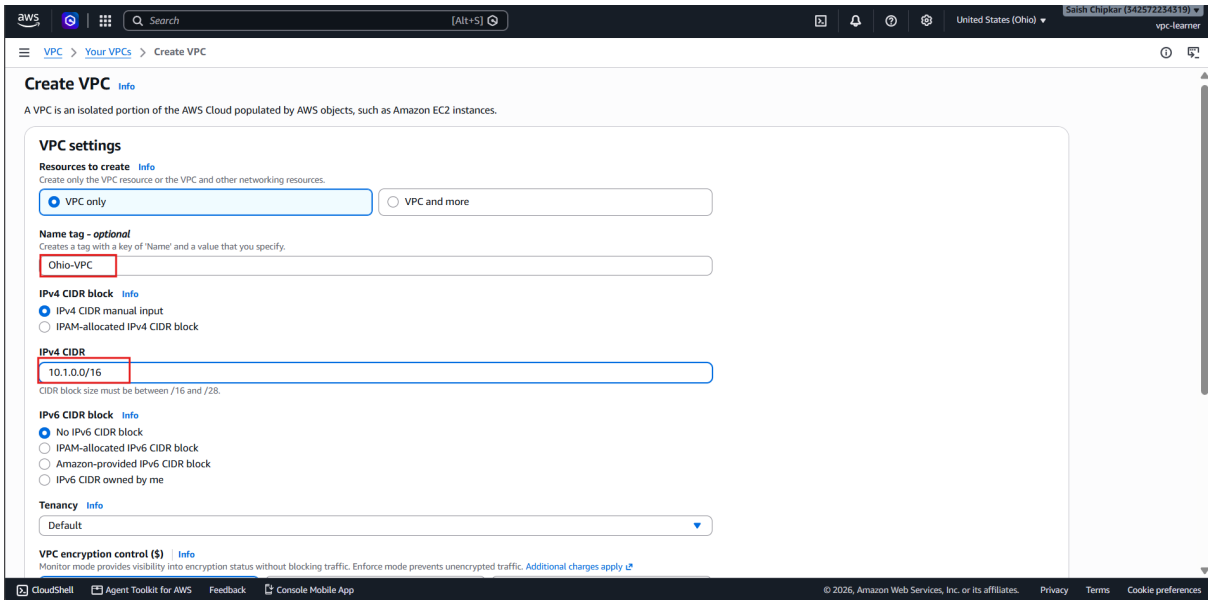


Figure 3: Entering the Ohio VPC name and CIDR block

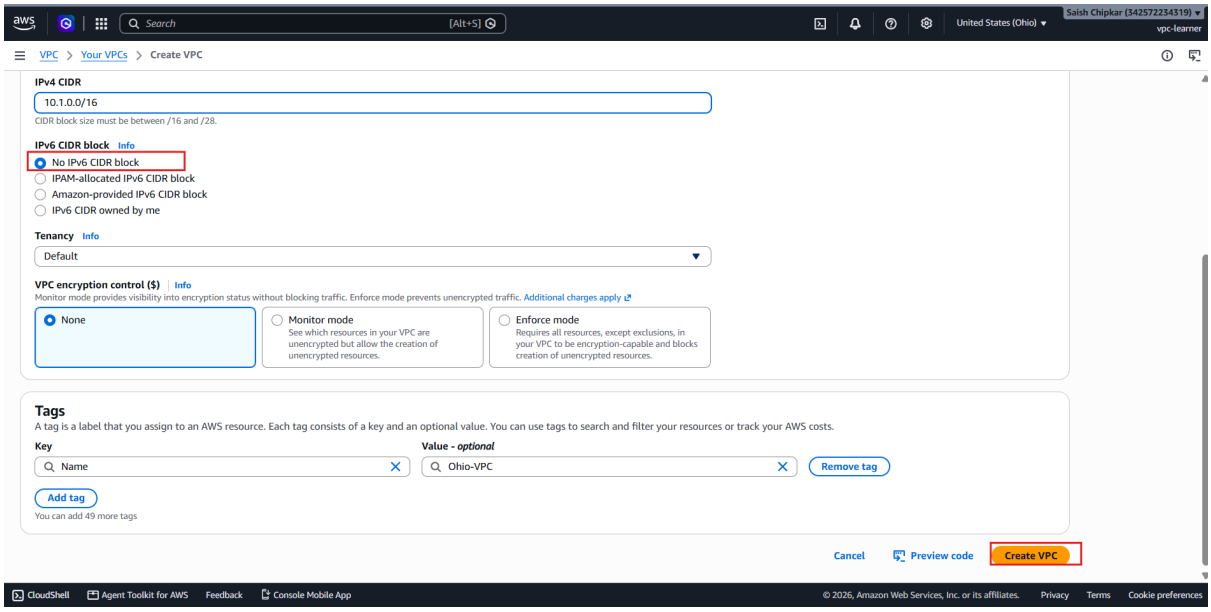


Figure 4: Reviewing the Ohio VPC settings

4.3 Step 3: Create the Ohio Public Subnet

Navigate to **VPC** → **Subnets** and click **Create subnet**.

Select `Ohio-VPC` and enter:

- Subnet name: `Public-Subnet`
- Availability Zone: `us-east-2a`
- IPv4 subnet CIDR block: `10.1.0.0/24`

Click **Create subnet**.

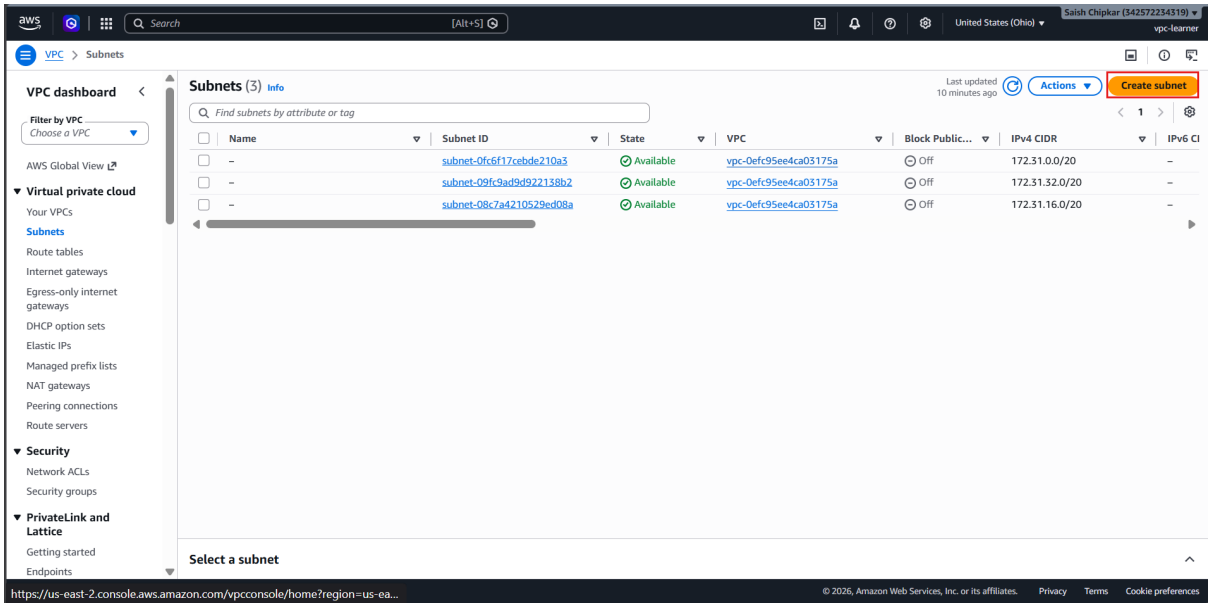


Figure 5: Selecting Ohio-VPC and configuring the subnet

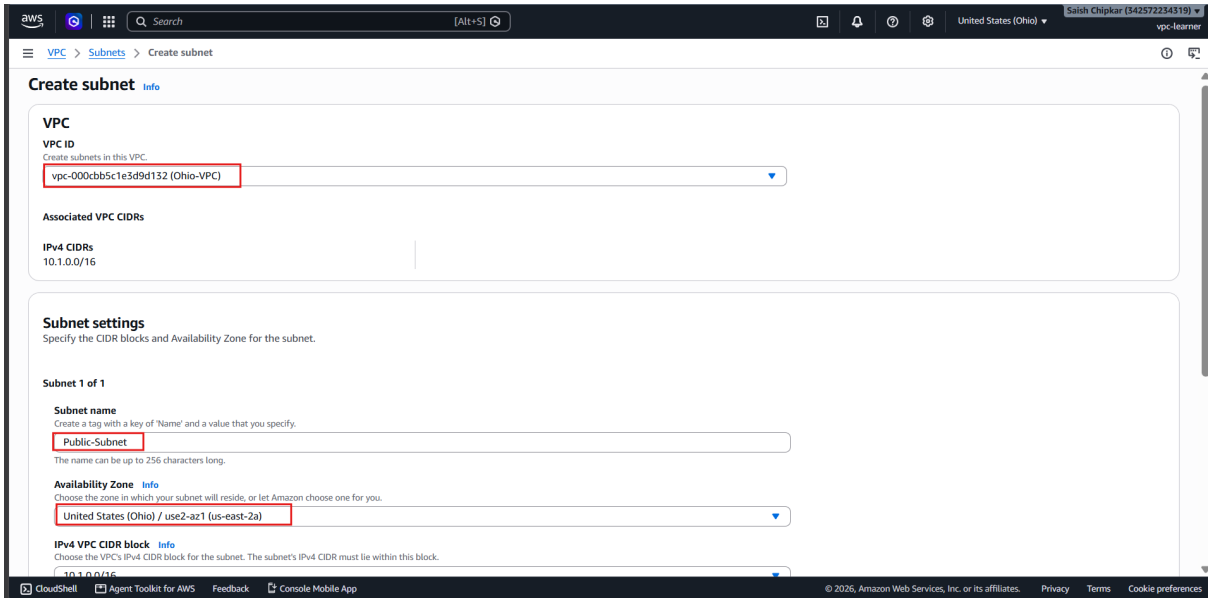


Figure 6: Entering the Ohio subnet CIDR block

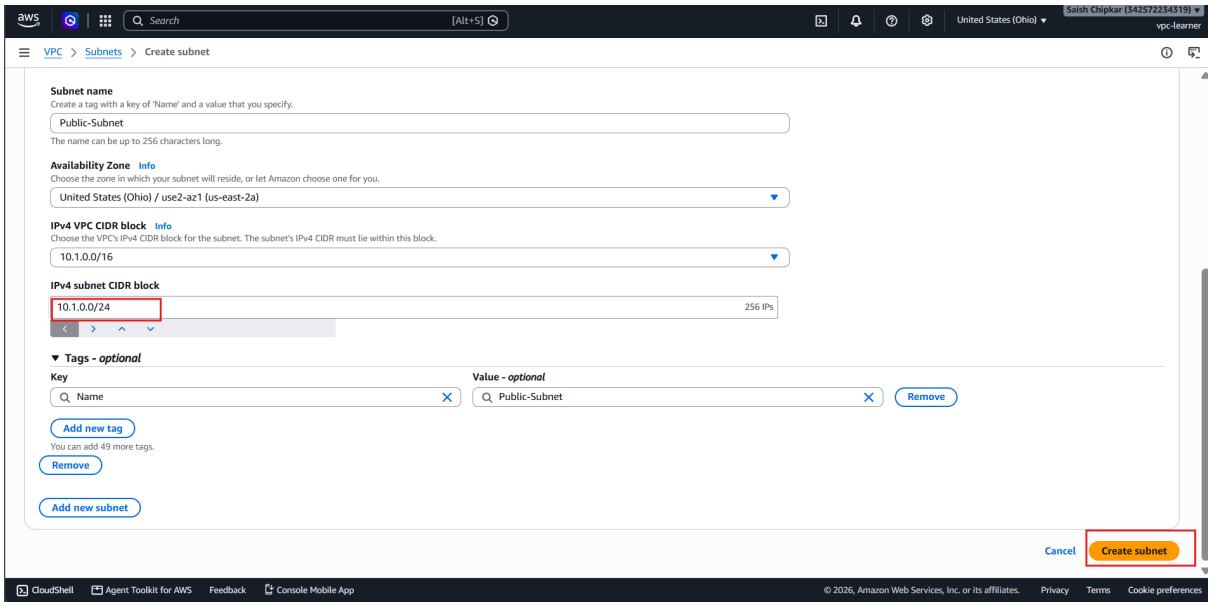


Figure 7: Successful creation of the Ohio subnet

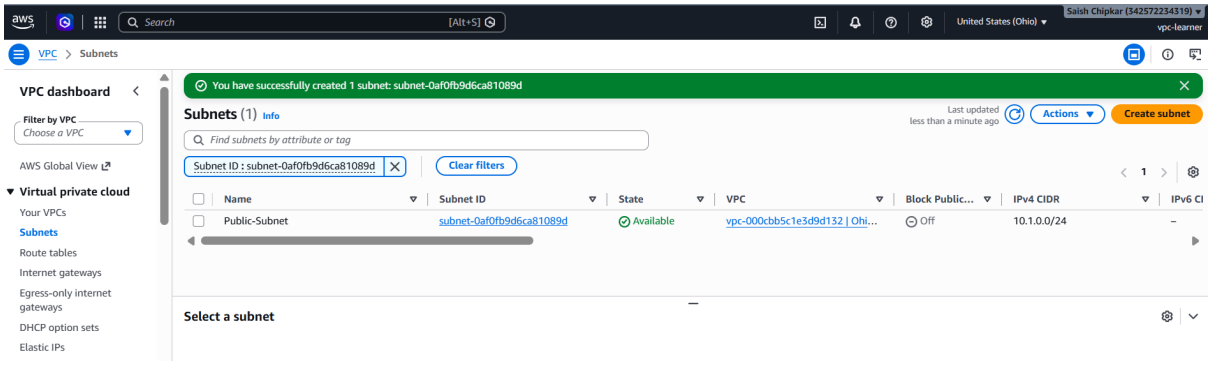


Figure 8: Sucessfull Creation of the Subnet

4.4 Step 4: Switch to the Mumbai Region

Open the AWS region selector and choose Asia Pacific (Mumbai), identified by `ap-south-1`.

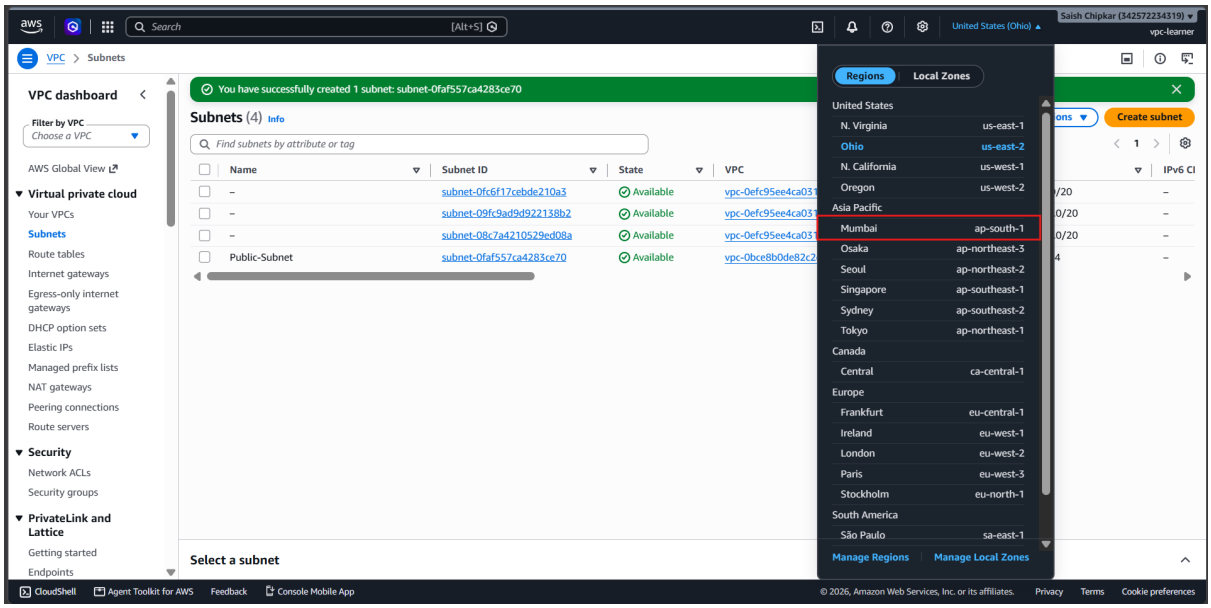


Figure 9: Selecting the Mumbai region

4.5 Step 5: Create the Mumbai VPC

Navigate to **Your VPCs** and click **Create VPC**. Select **VPC only**.

Enter the following configuration:

- Name tag: `Mumbai-VPC`
- IPv4 CIDR block: `10.0.128.0/17`
- IPv6 CIDR block: No IPv6 CIDR block
- Tenancy: Default

Click **Create VPC**.

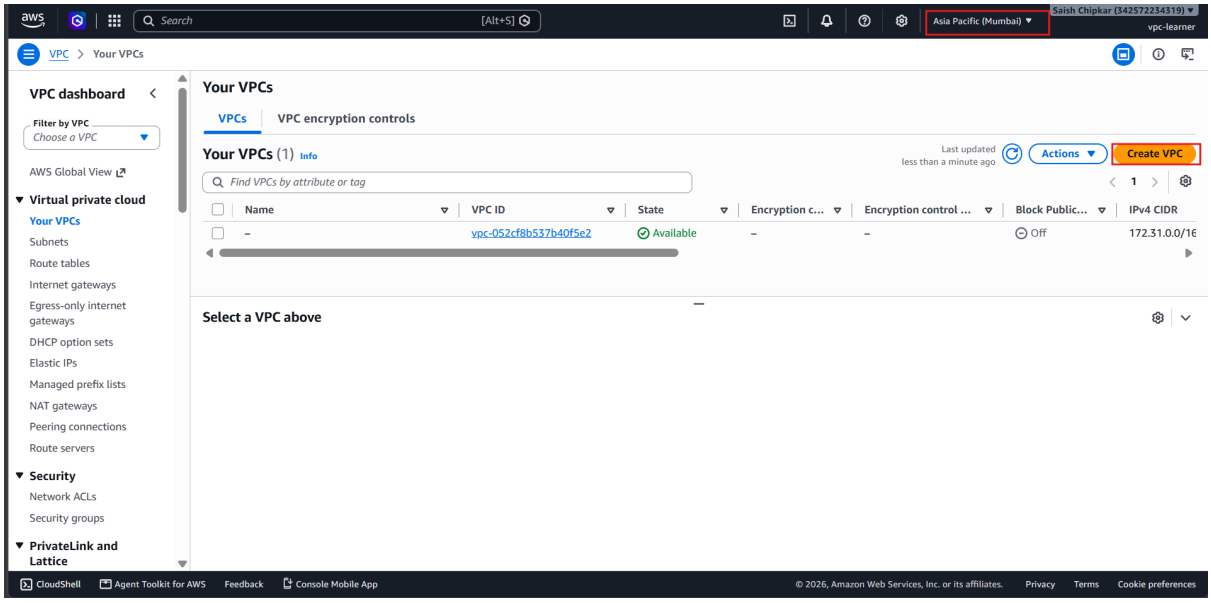


Figure 10: Opening the Create VPC page in Mumbai

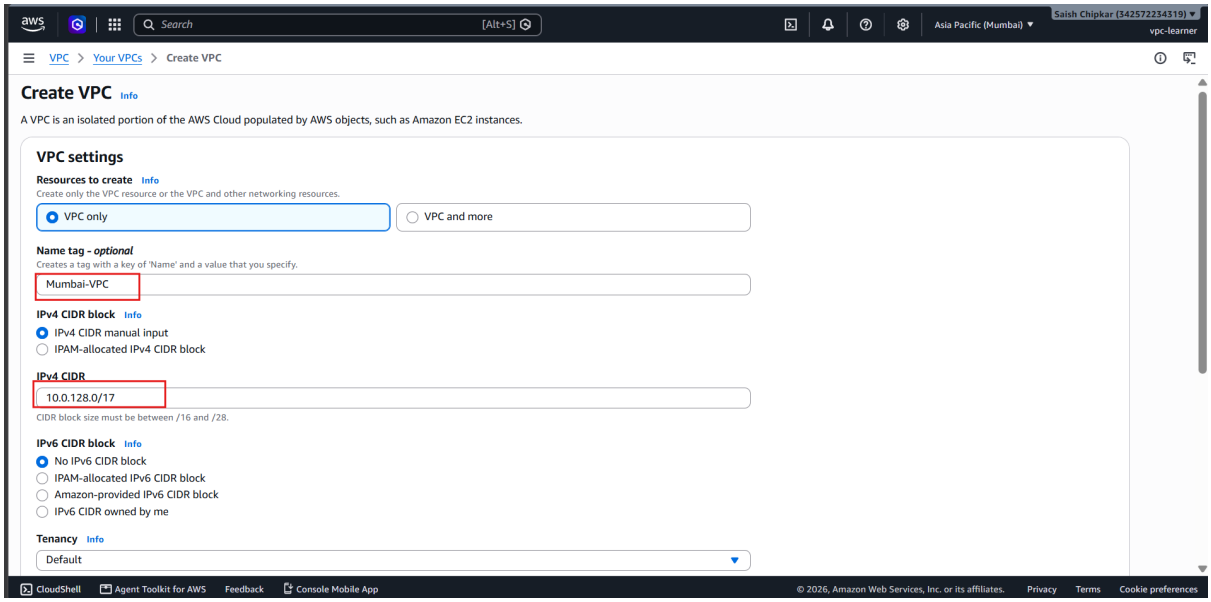


Figure 11: Entering the Mumbai VPC configuration

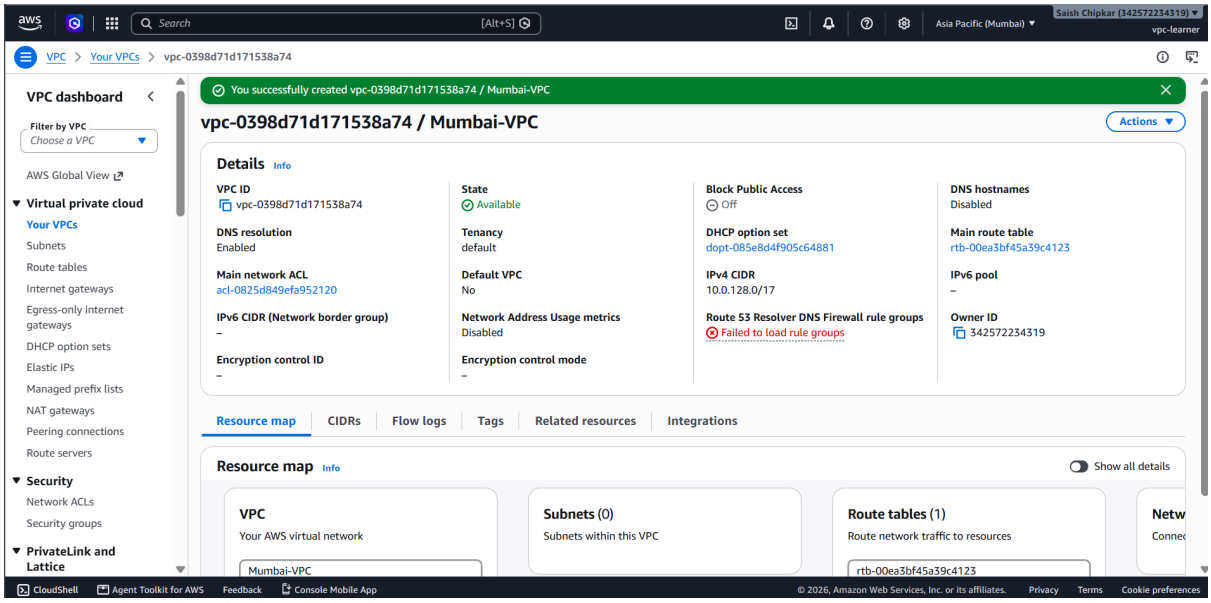


Figure 12: Successful creation of Mumbai-VPC

4.6 Step 6: Create the Mumbai Public Subnet

Navigate to **Subnets** and click **Create subnet**. Select `Mumbai-VPC` and enter:

- Subnet name: `Public-Subnet`
- Availability Zone: `ap-south-1a`
- IPv4 subnet CIDR block: `10.0.128.0/24`

Click **Create subnet**.

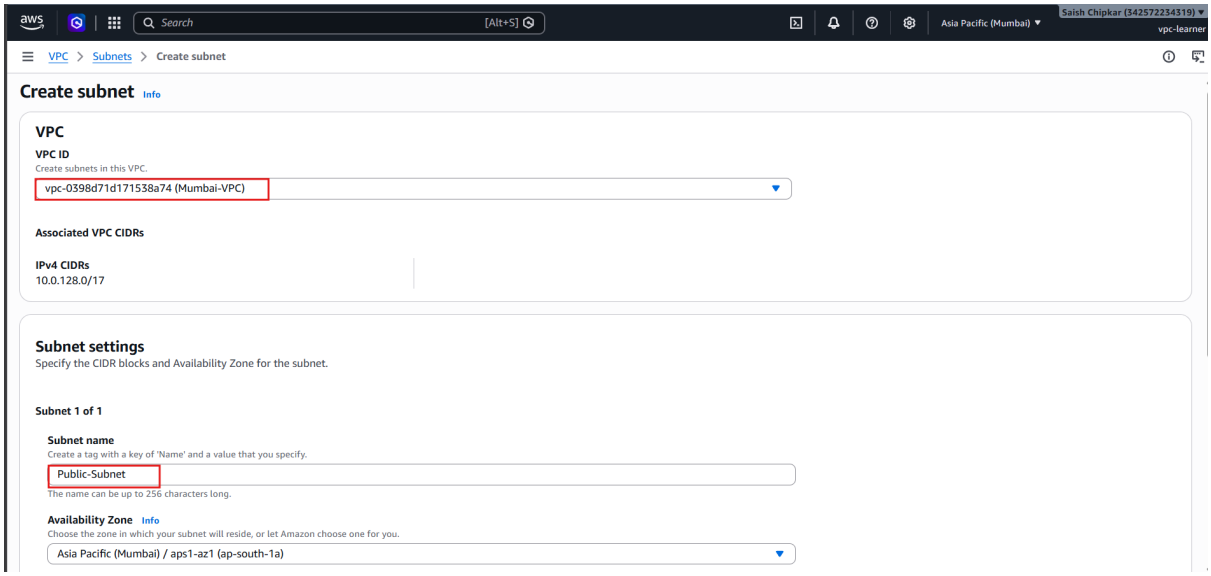


Figure 13: Selecting Mumbai-VPC for subnet creation

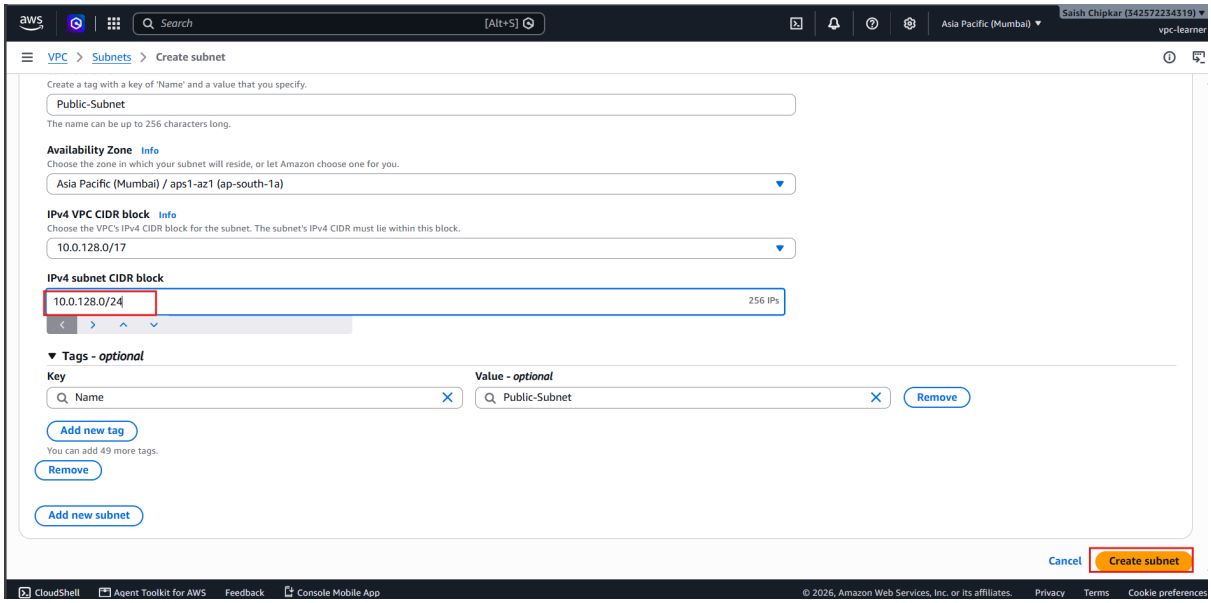


Figure 14: Configuring the Mumbai subnet CIDR block

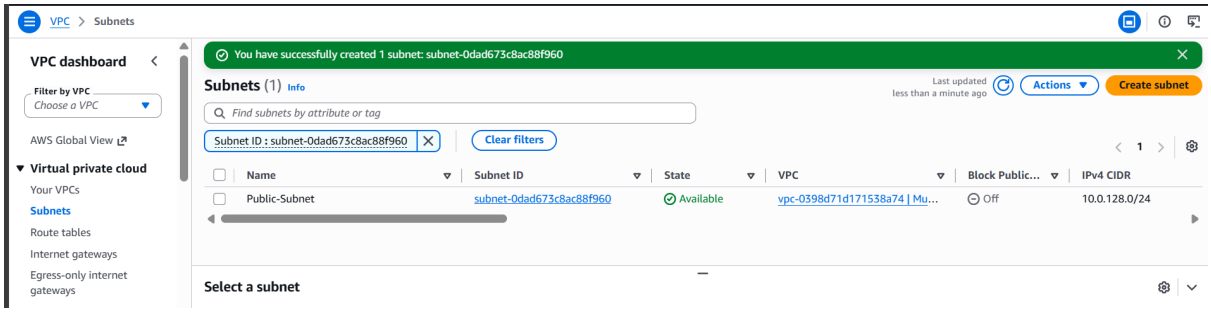


Figure 15: Successful creation of the Mumbai subnet

4.7 Step 7: Create the Ohio Internet Gateway

Switch the AWS region back to Ohio (`us-east-2`).

Navigate to **Internet Gateways** and click **Create internet gateway**. Enter the name tag `Ohio-IGW` and click **Create internet gateway**.

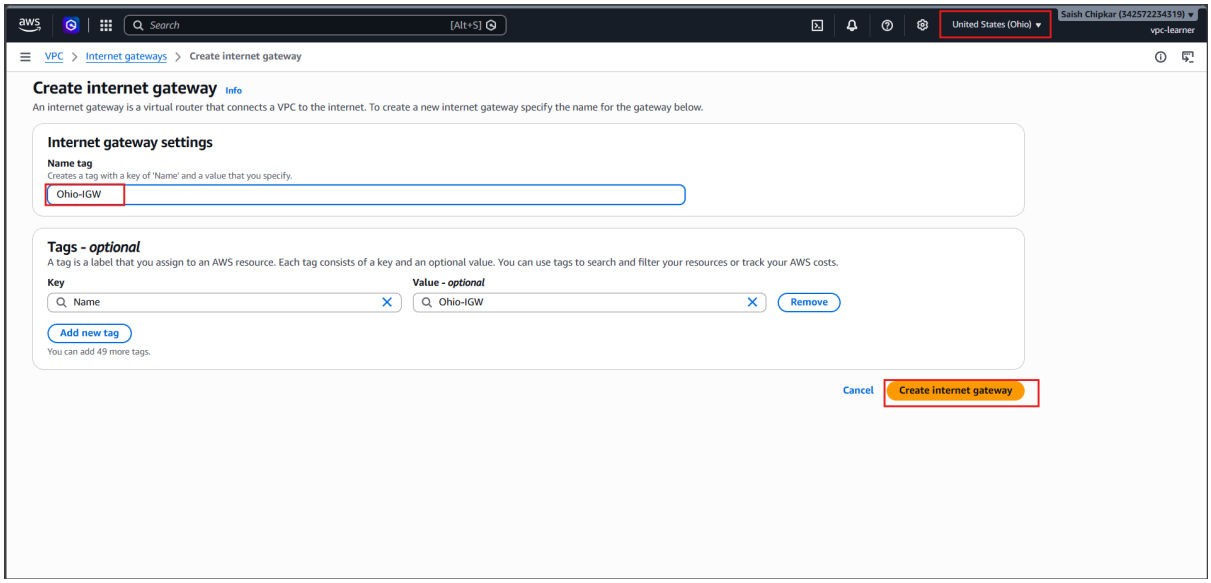


Figure 16: Creating the Ohio Internet Gateway

4.8 Step 8: Attach the Ohio Internet Gateway

Select `Ohio-IGW`. Choose **Actions** → **Attach to a VPC**.

Select `Ohio-VPC` and click **Attach internet gateway**.

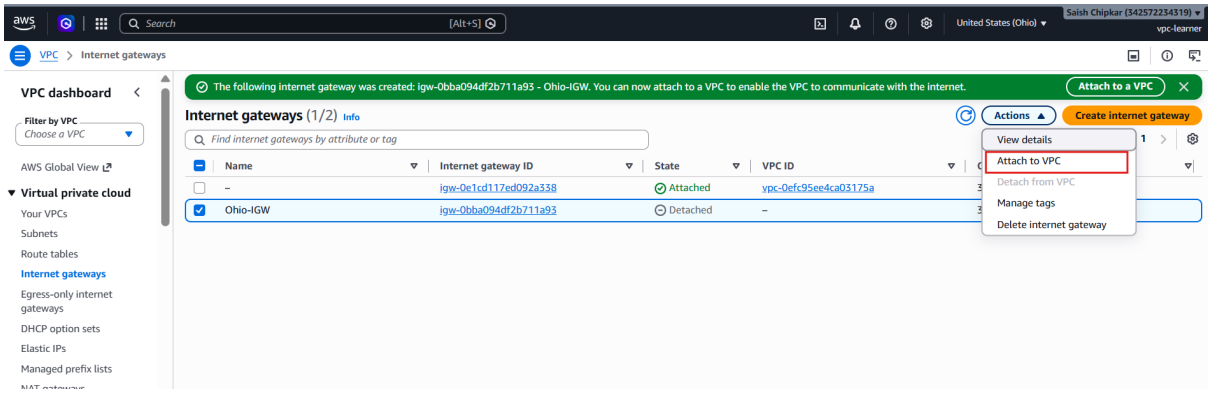


Figure 17: Opening the Attach to VPC action

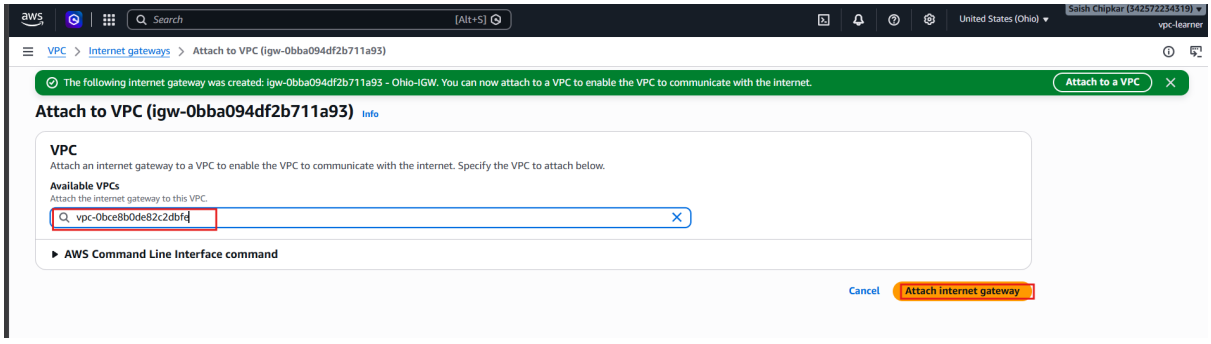


Figure 18: Selecting Ohio-VPC for gateway attachment

4.9 Step 9: Create the Mumbai Internet Gateway

Switch the AWS region to Mumbai (`ap-south-1`).

Navigate to **Internet Gateways** and click **Create internet gateway**. Enter the name tag `Mumbai-IGW` and click **Create internet gateway**.

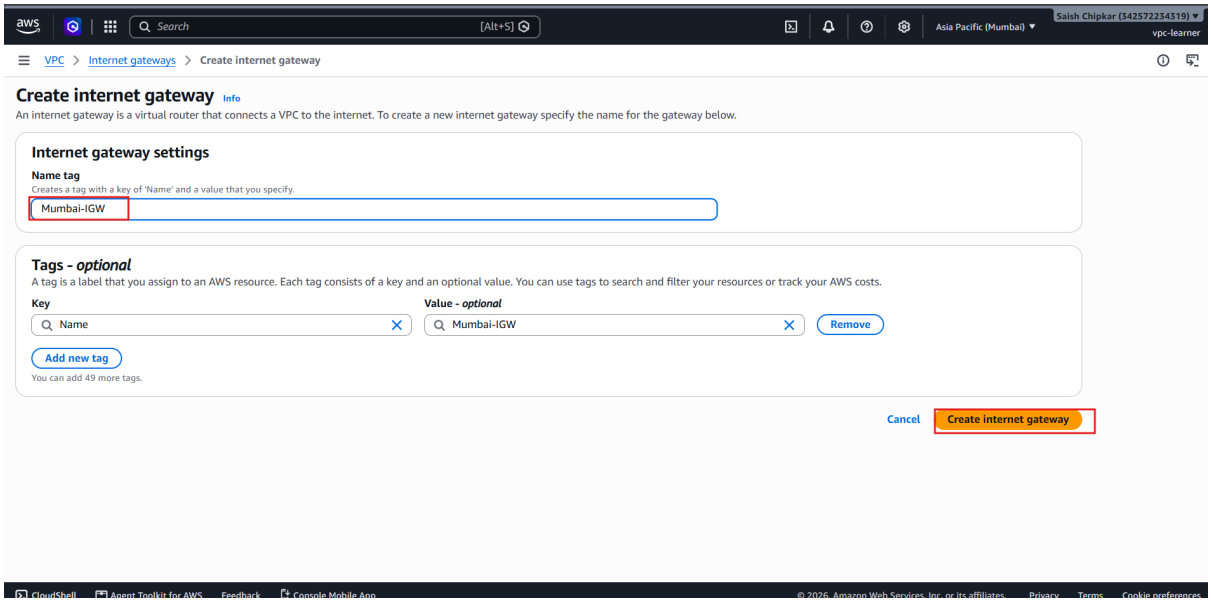


Figure 19: Creating the Mumbai Internet Gateway

4.10 Step 10: Attach the Mumbai Internet Gateway

Select `Mumbai-IGW` and choose **Actions** → **Attach to a VPC**.

Select `Mumbai-VPC` and click **Attach internet gateway**.

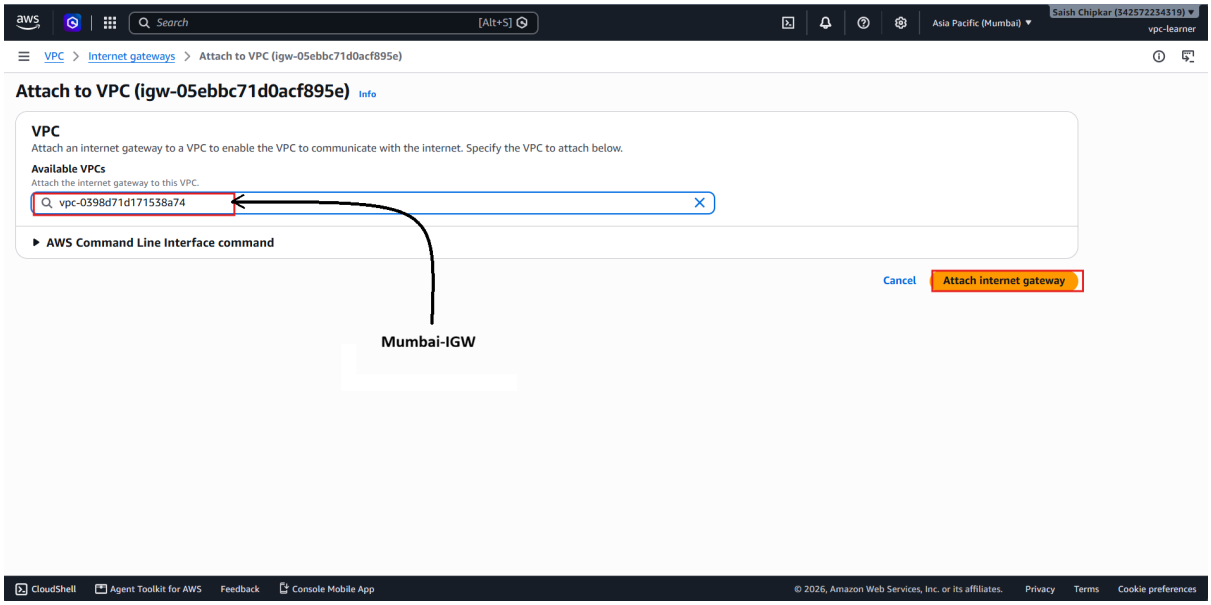


Figure 20: Selecting Mumbai-VPC for gateway attachment

4.11 Step 11: Open Peering Connections

In the Ohio region (`us-east-2`), navigate to **VPC → Peering connections**. Click **Create peering connection** to establish private connectivity between Ohio-VPC and Mumbai-VPC.

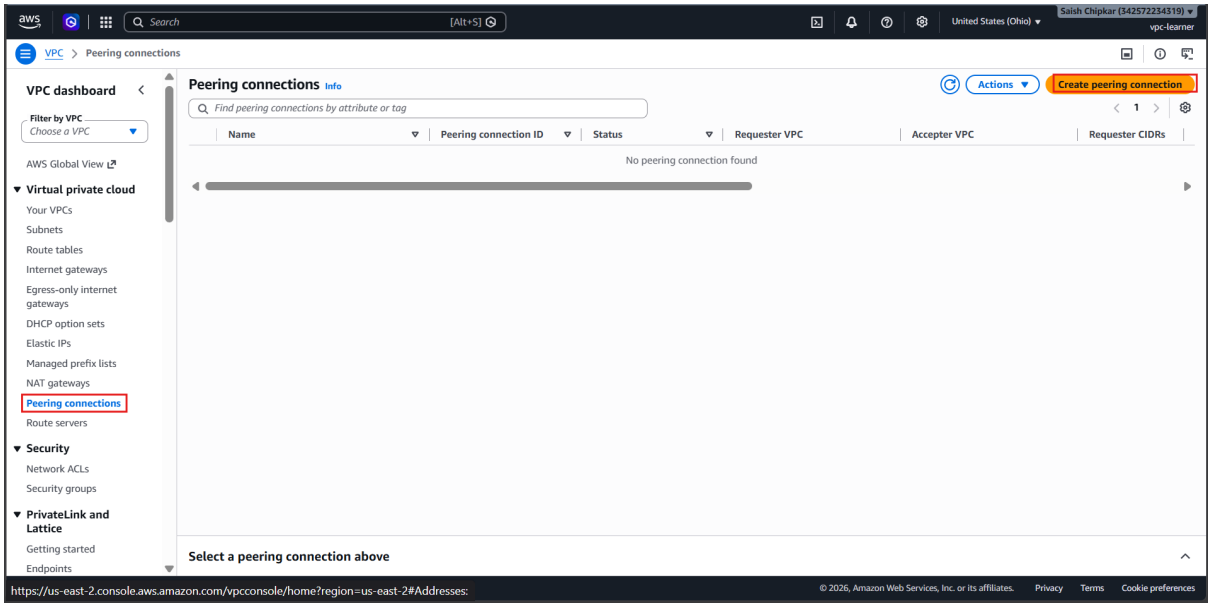


Figure 21: Opening the VPC Peering Connections dashboard

4.12 Step 12: Configure the Peering Connection

Enter the following configuration:

- Name: `Ohio-PC`
- VPC ID (Requester): Select `Ohio-VPC`
- Account: My account
- Region: Another Region
- Accepter Region: Asia Pacific (Mumbai)
- VPC ID (Accepter): Enter the ID of `Mumbai-VPC`

The requester VPC uses the CIDR block `10.1.0.0/16`, while the accepter VPC uses `10.0.128.0/17`.

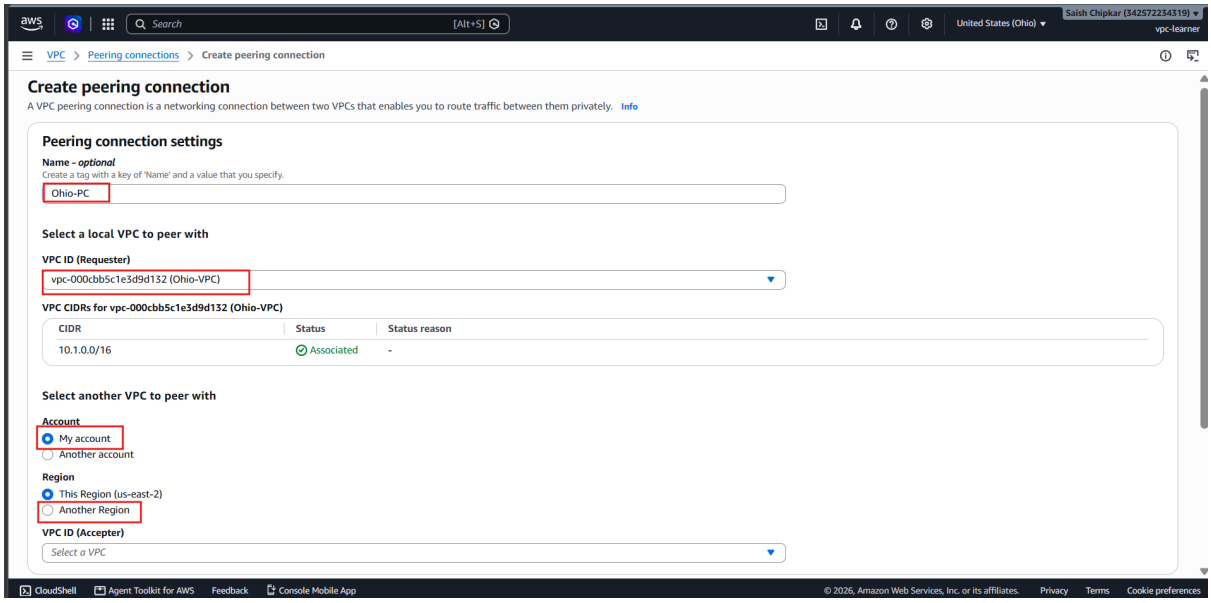


Figure 22: Configuring the requester VPC and peering settings

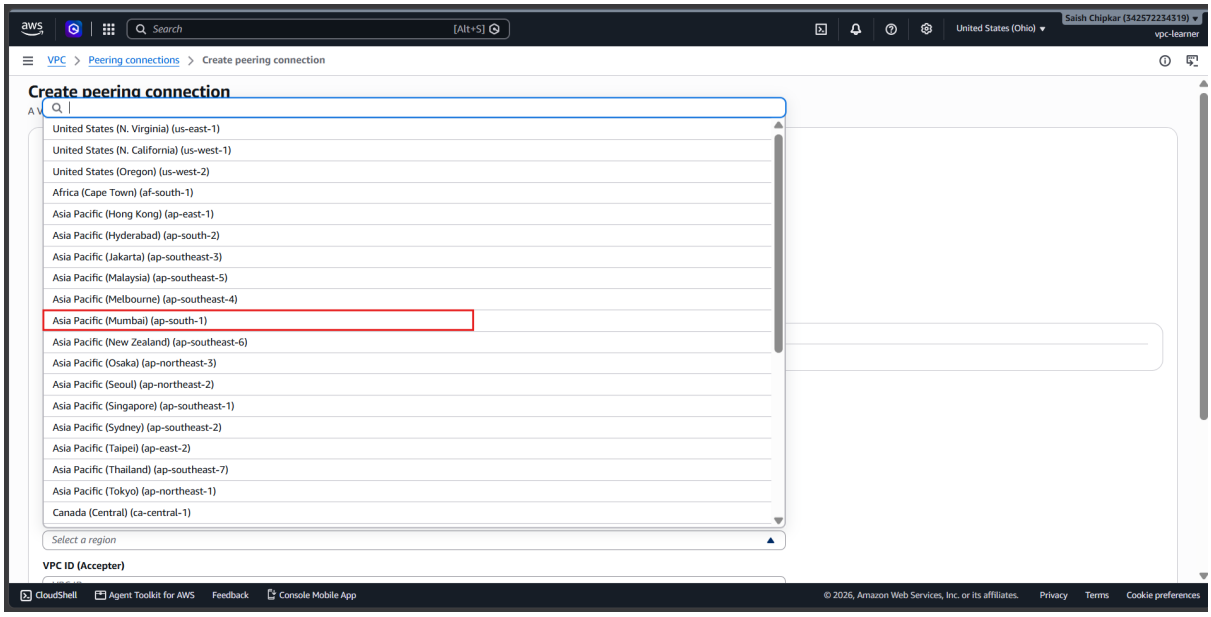


Figure 23: Selecting the Mumbai region as the accepter region

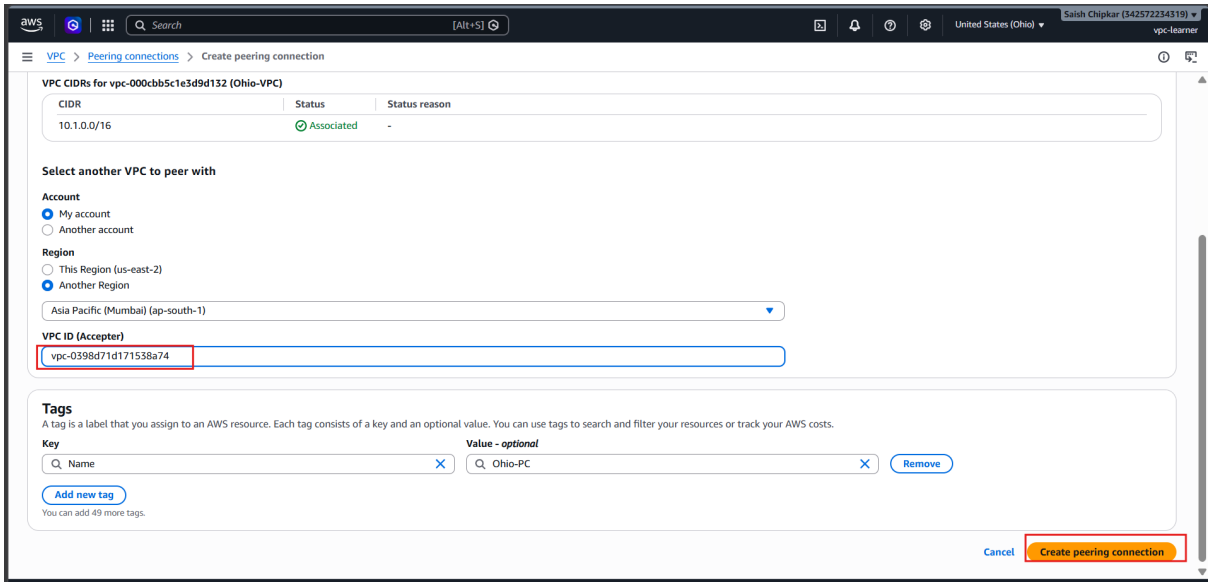


Figure 24: Entering the Mumbai VPC ID and creating the peering connection

4.13 Step 13: Verify the Peering Request

After clicking **Create peering connection**, AWS creates the peering request. The connection initially remains in the pending acceptance state because the accepter VPC must approve the request.

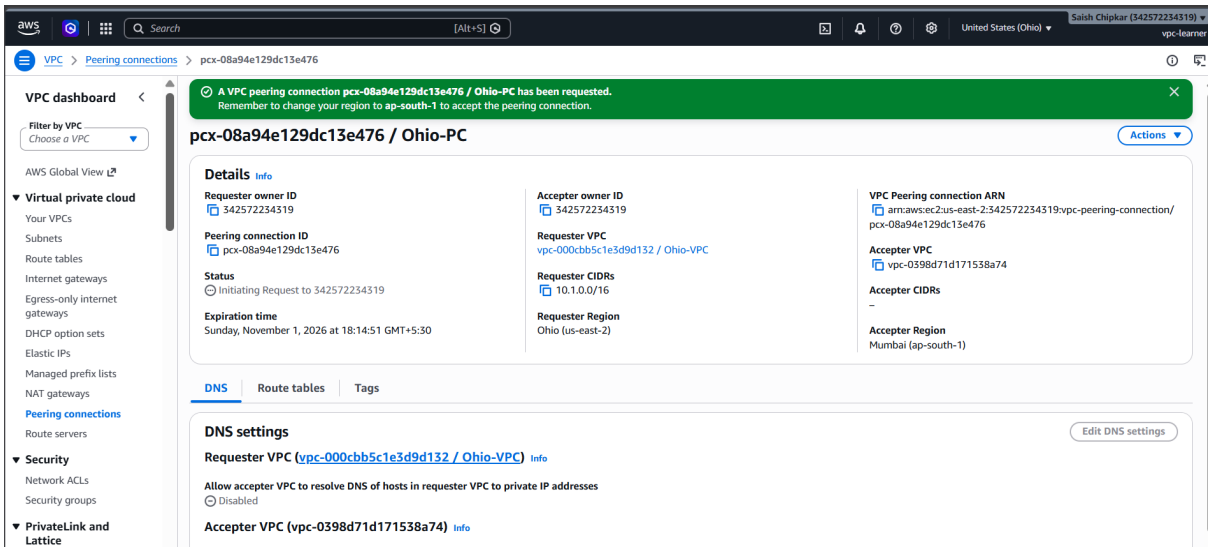


Figure 25: Peering connection request created successfully

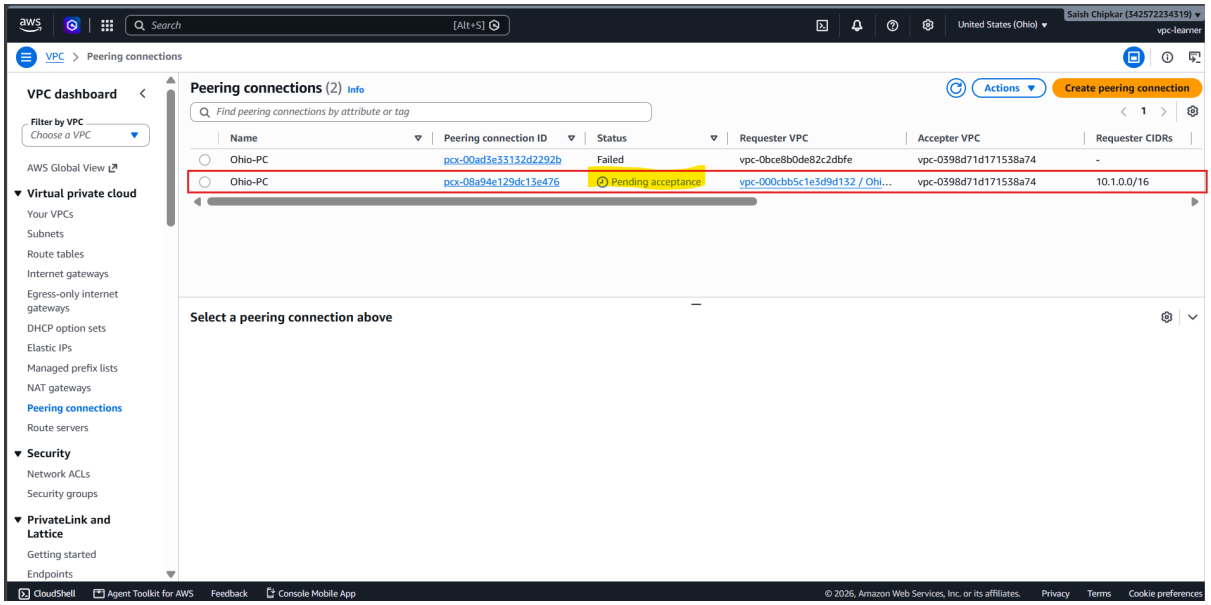


Figure 26: Peering connection in the pending acceptance state

4.14 Step 14: Accept the Peering Request in Mumbai

Switch the AWS region to Asia Pacific (Mumbai) (`ap-south-1`).

Navigate to **VPC → Peering connections**. Select the pending connection and choose **Actions → Accept request**.

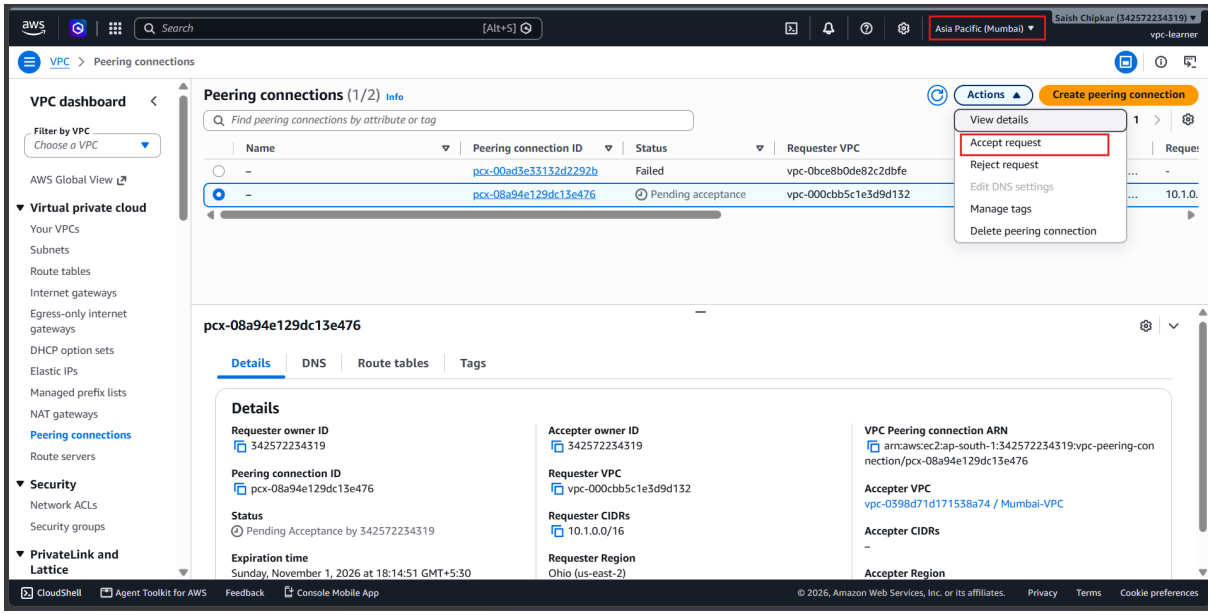


Figure 27: Selecting the Accept request action in Mumbai

Review the requester and accepter VPC details, then click **Accept request** to confirm.

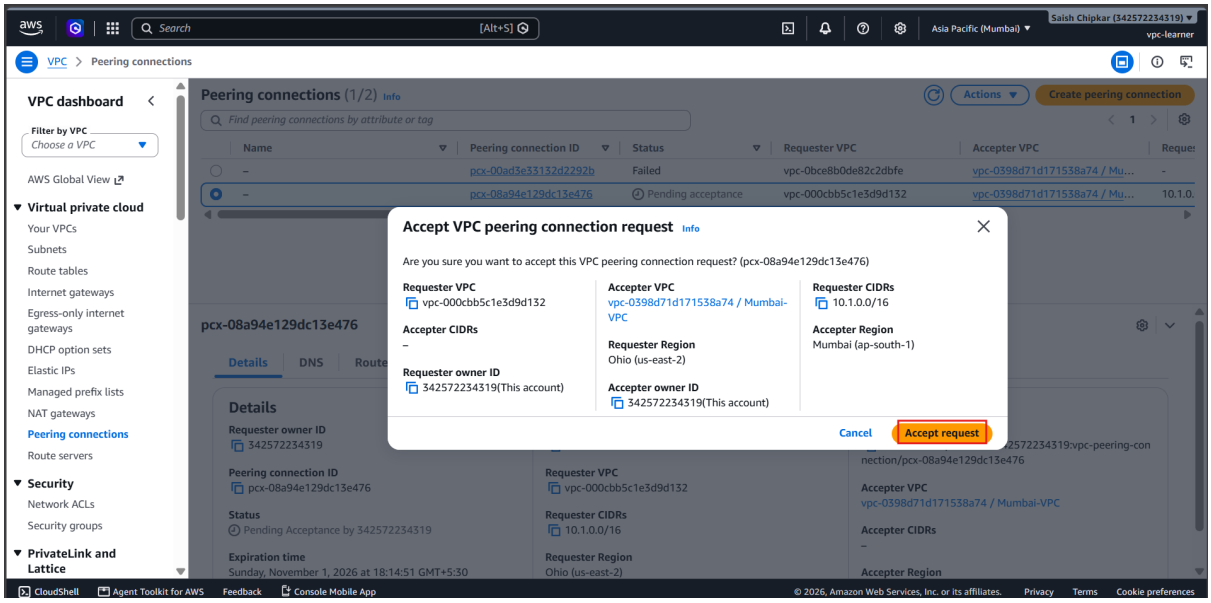


Figure 28: Confirming the VPC peering request

4.15 Step 15: Verify the Peering Connection

AWS establishes the VPC peering connection after the request is accepted. Verify that the connection is established in the Mumbai region.

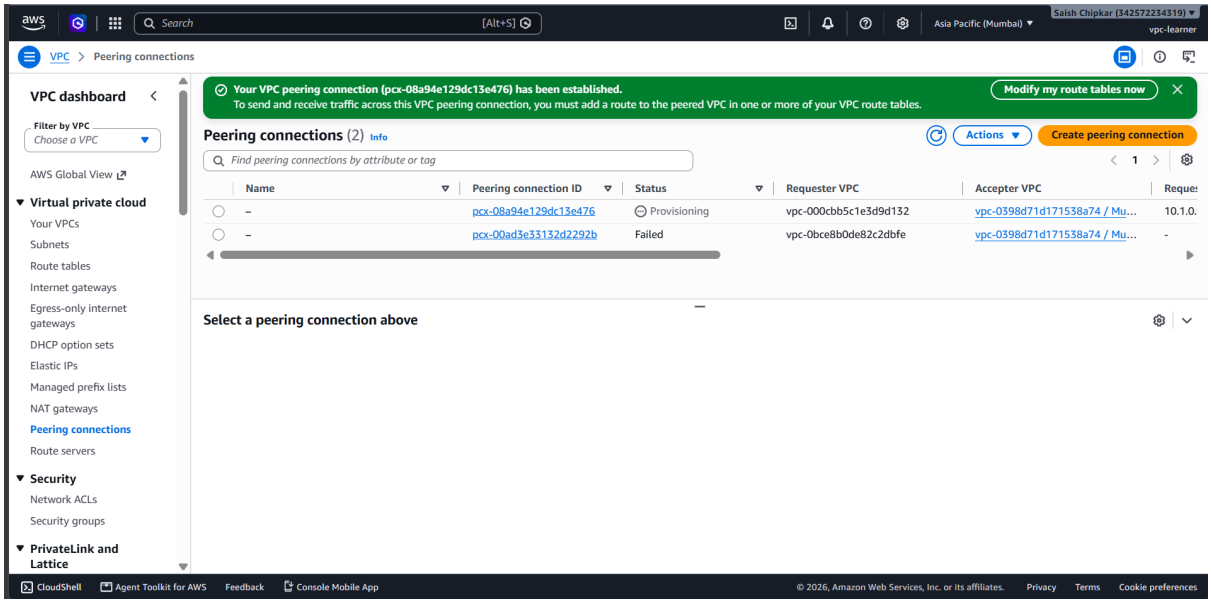


Figure 29: VPC peering connection established successfully

4.16 Step 16: Create a Custom Route Table in Ohio

Switch back to the Ohio region (`us-east-2`).

Navigate to **VPC** → **Route tables** and click **Create route table**.

Enter the following details:

- Name: `Ohio-RT`
- VPC: `Ohio-VPC`

Click **Create route table**.

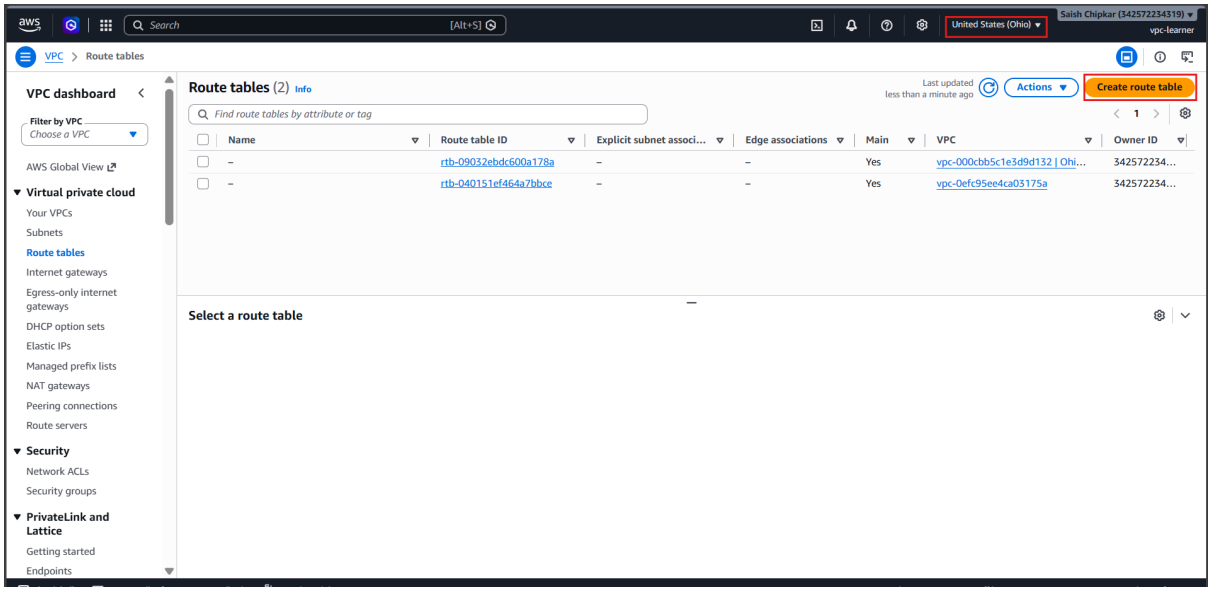


Figure 30: Opening the route tables dashboard in Ohio

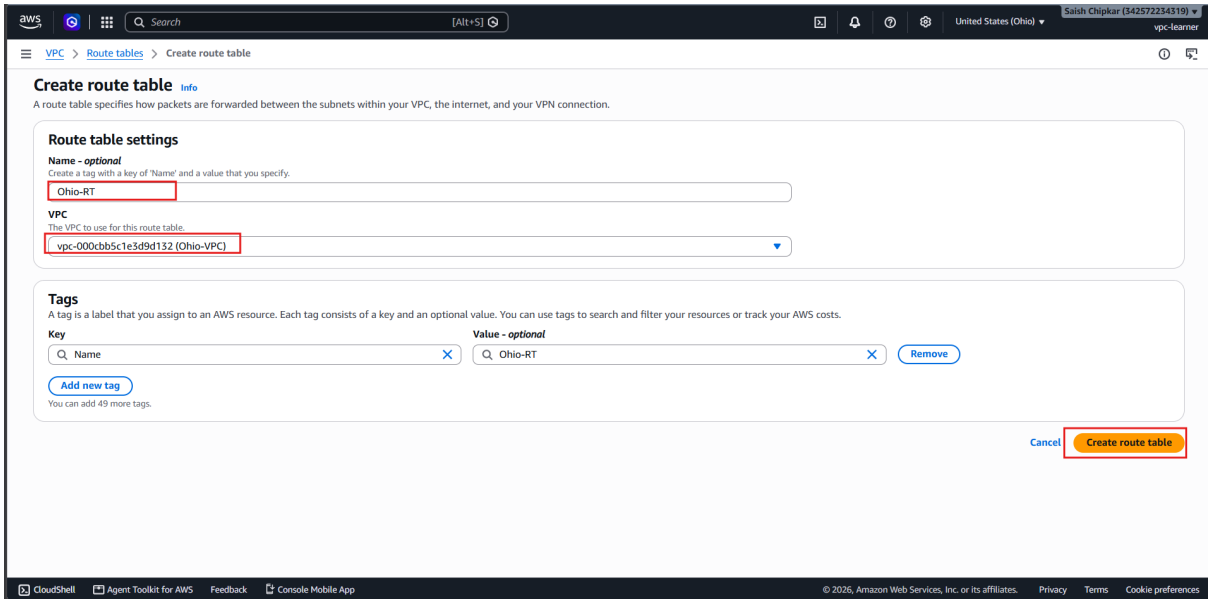


Figure 31: Creating the Ohio custom route table

4.17 Step 17: Configure Routes in Ohio-RT

Select `Ohio-RT` and open the **Routes** tab. Click **Edit routes** and add the following routes:

- Destination: `0.0.0.0/0`
  Target: Ohio Internet Gateway (`Ohio-IGW`)
- Destination: `10.0.128.0/17`
  Target: VPC peering connection (`Ohio-PC`)

The first route allows internet-bound traffic to use the Internet Gateway. The second route directs traffic destined for the Mumbai VPC through the peering connection.

Keep the automatically created local route `10.1.0.0/16` unchanged.

Click **Save changes**.

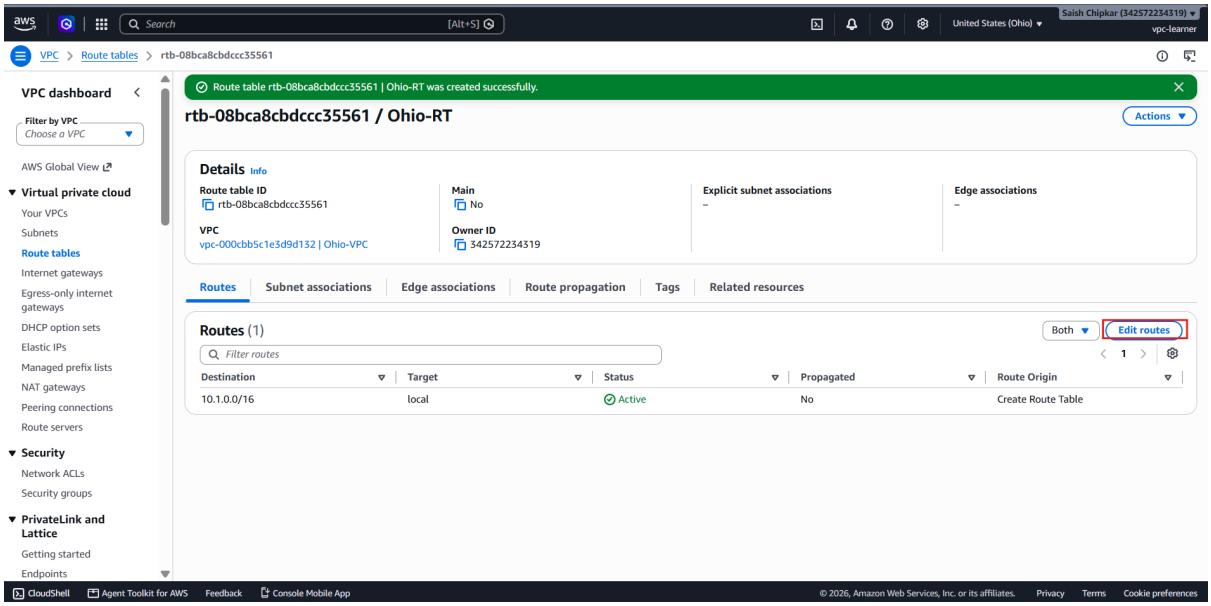


Figure 32: Adding the Internet Gateway and peering routes in Ohio

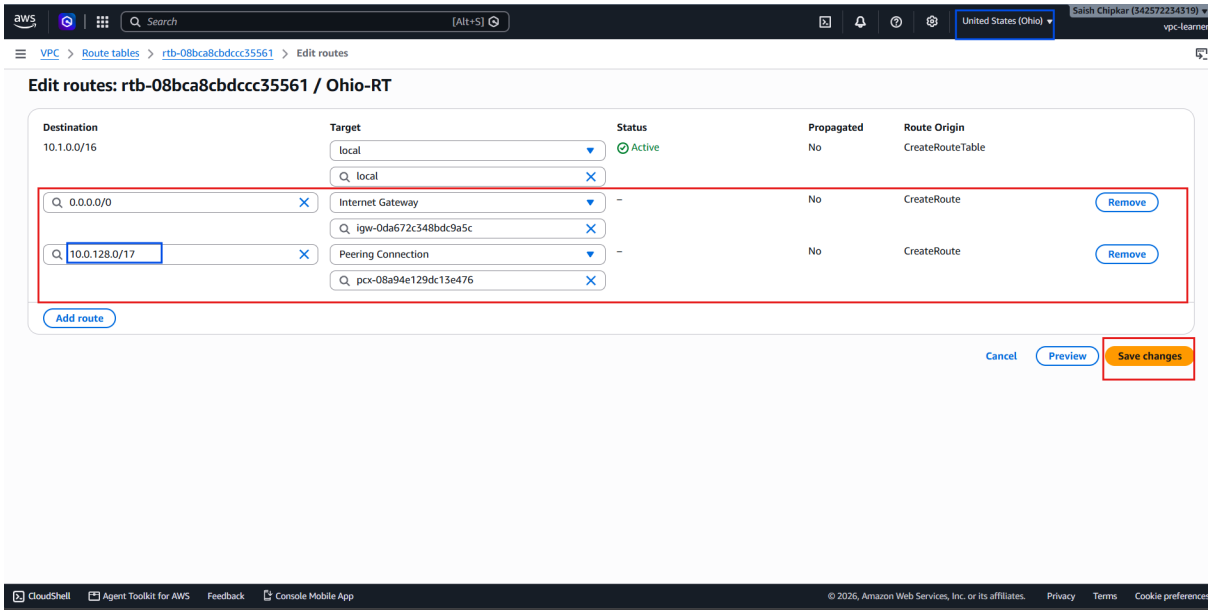


Figure 33: Ohio route table updated successfully

4.18 Step 18: Associate the Ohio Public Subnet

Open the **Subnet associations** tab for `Ohio-RT`. Click **Edit subnet associations** and select the Ohio `Public-Subnet`, whose CIDR block is `10.1.0.0/24`.

Click **Save associations**.

This explicitly associates the Ohio public subnet with the custom route table.

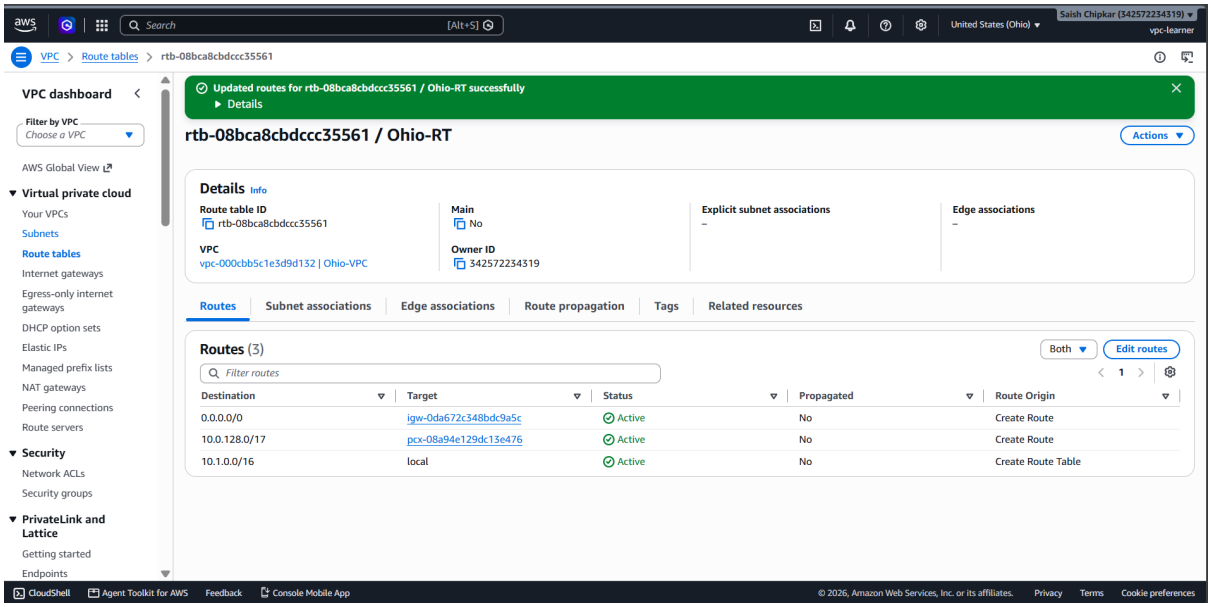


Figure 34: Selecting the Ohio public subnet for association

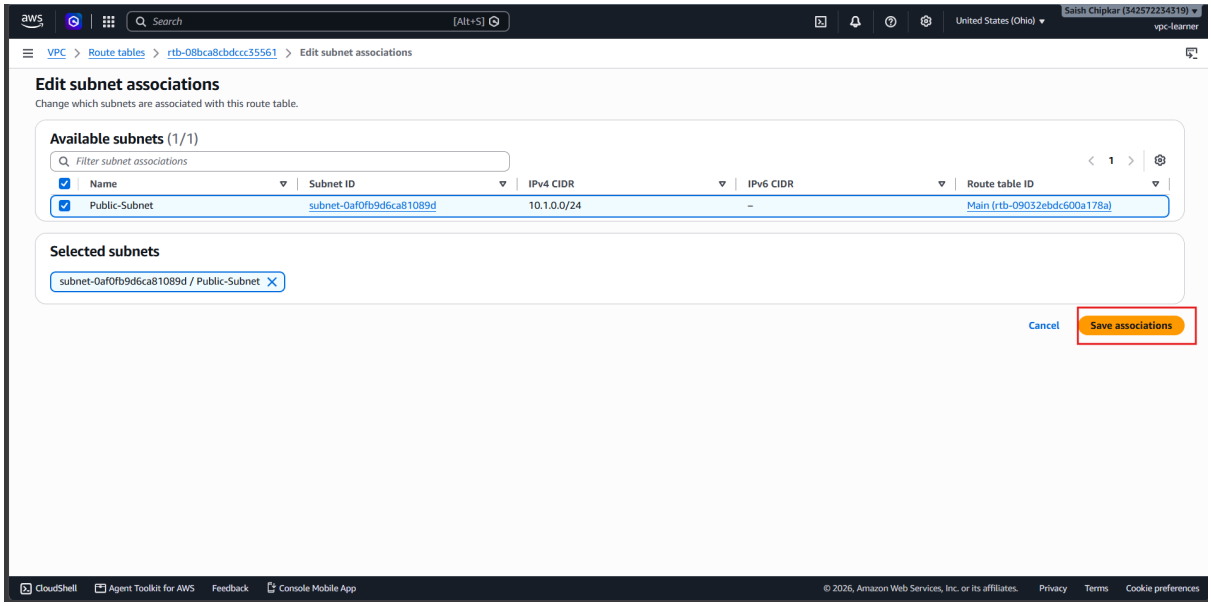


Figure 35: Ohio public subnet associated with Ohio-RT

4.19 Step 19: Create a Custom Route Table in Mumbai

Switch to the Mumbai region (`ap-south-1`).

Navigate to **VPC → Route tables** and click **Create route table**.

Enter the following details:

- Name: `Mumbai-RT`
- VPC: `Mumbai-VPC`

Click **Create route table**.

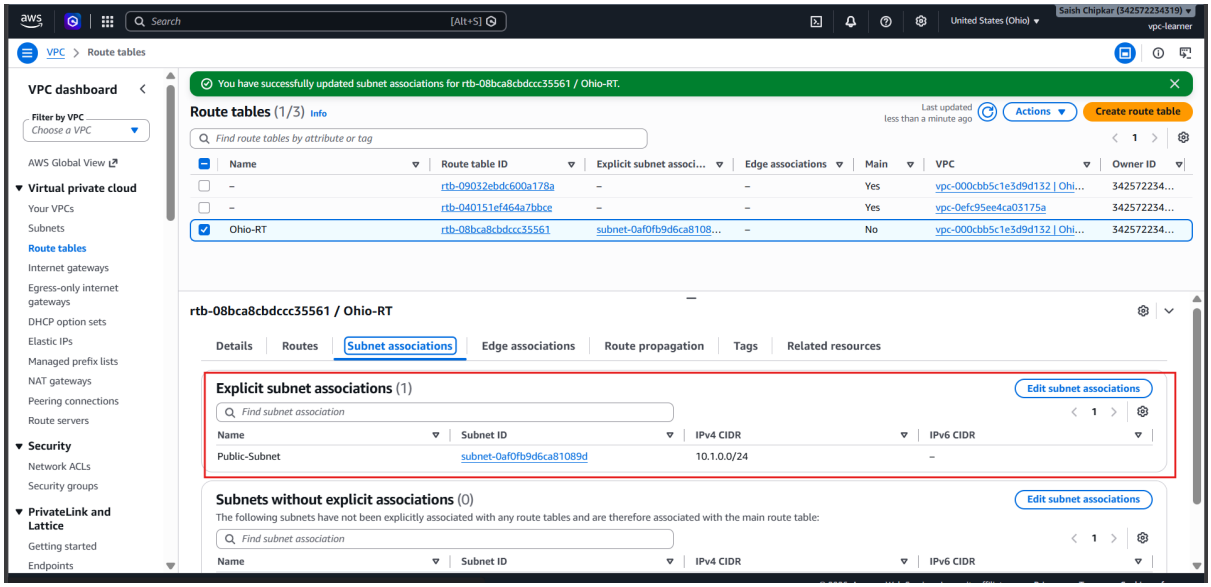


Figure 36: Creating the Mumbai custom route table

4.20 Step 20: Configure Routes in Mumbai-RT

Select `Mumbai-RT` and open the **Routes** tab. Click **Edit routes** and add the following routes:

- Destination: `0.0.0.0/0`
  Target: Mumbai Internet Gateway (`Mumbai-IGW`)
- Destination: `10.1.0.0/16`
  Target: VPC peering connection (`Ohio-PC`)

The first route directs internet-bound traffic to the Mumbai Internet Gateway. The second route directs traffic destined for Ohio-VPC through the peering connection.

Keep the automatically created local route `10.0.128.0/17` unchanged.

Click **Save changes**.

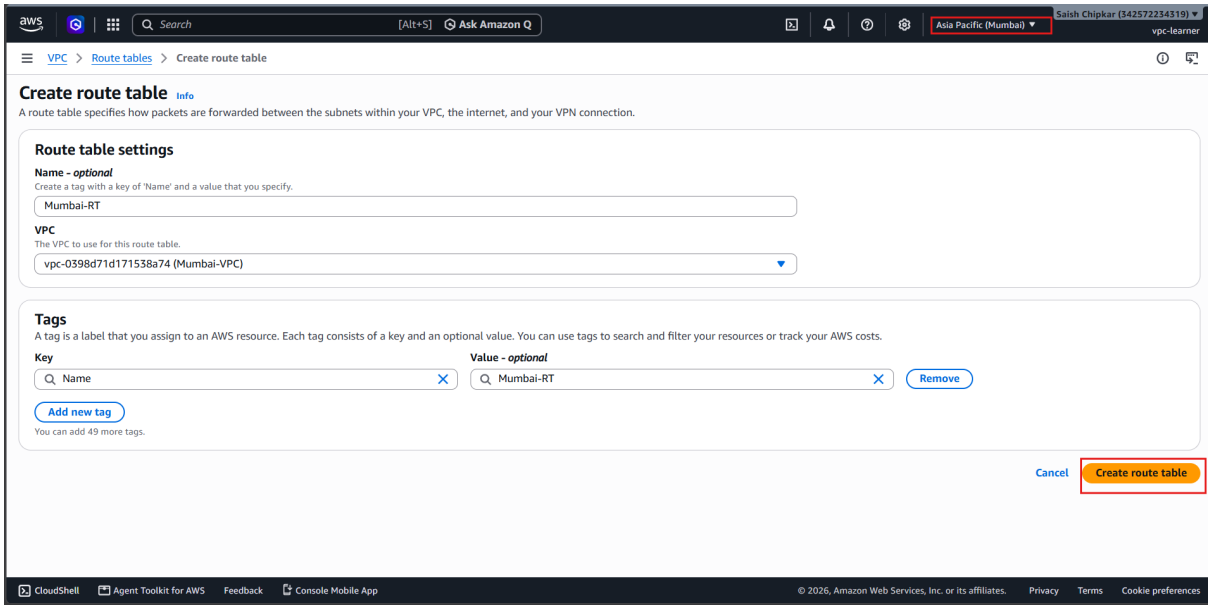


Figure 37: Mumbai route table configured with the peering route

4.21 Step 21: Associate the Mumbai Public Subnet

Open the **Subnet associations** tab for `Mumbai-RT`. Click **Edit subnet associations** and select the Mumbai `Public-Subnet`, whose CIDR block is `10.0.128.0/24`.

Click **Save associations** to complete the association.

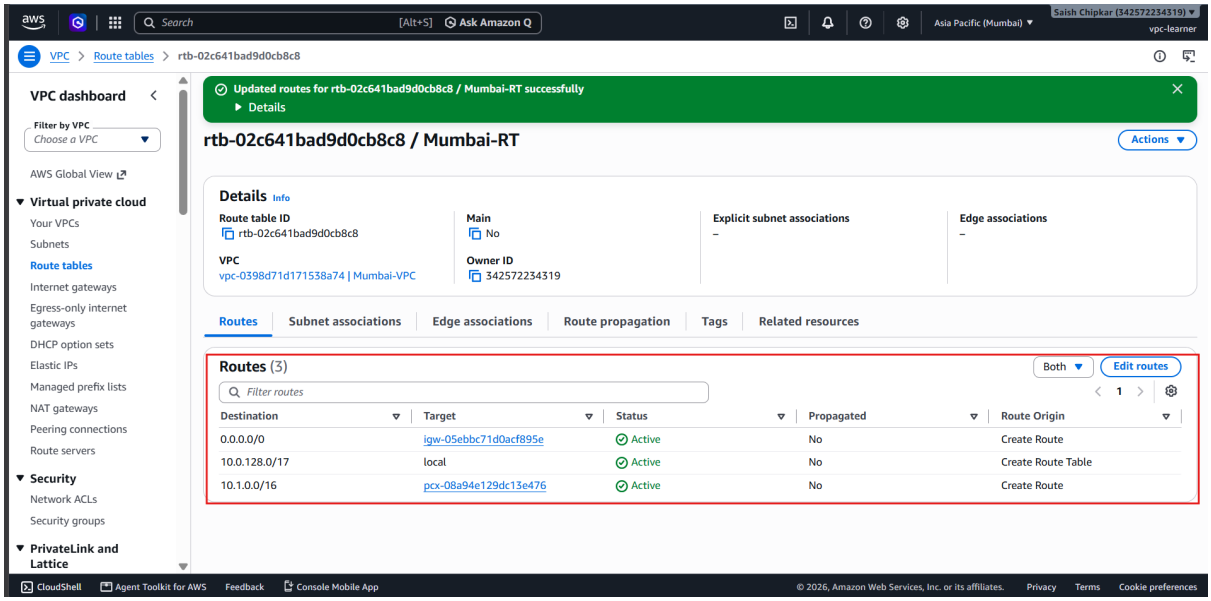


Figure 38: Saving the Mumbai subnet association

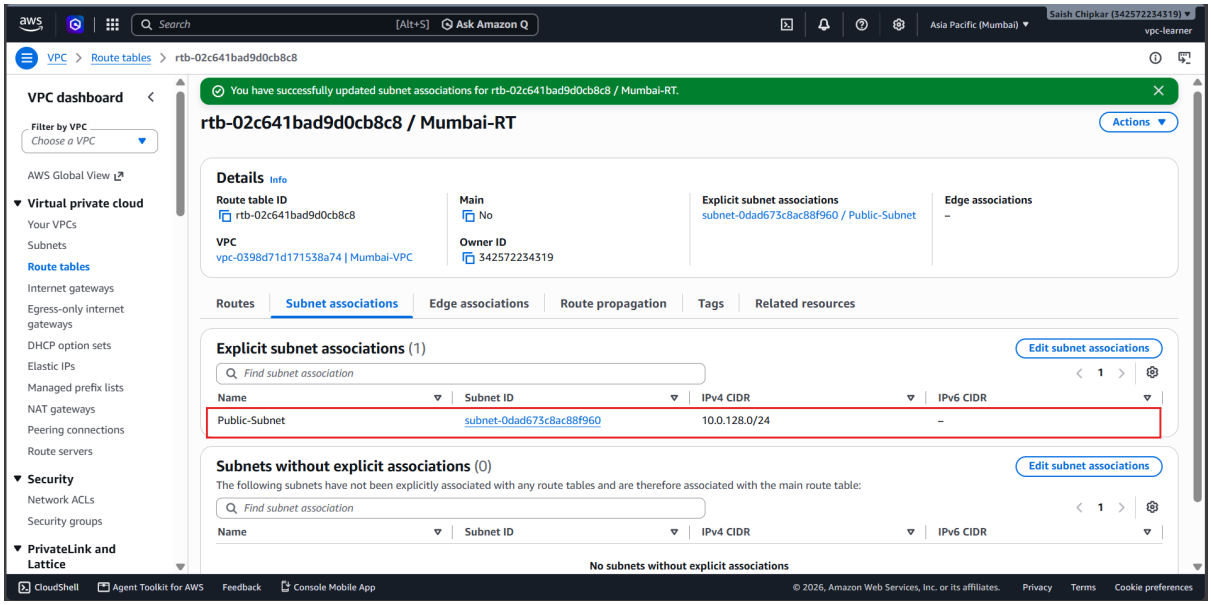


Figure 39: Mumbai public subnet associated with Mumbai-RT

5 Route Table Configuration Summary

Destination	Ohio-RT Target	Mumbai-RT Target
Local VPC CIDR	Local	Local
`0.0.0.0/0`	Ohio-IGW	Mumbai-IGW
`10.0.128.0/17`	Ohio-PC	–
`10.1.0.0/16`	–	Ohio-PC

Table 1: Routes configured in the Ohio and Mumbai route tables

5.1 Step 22: Launch the Mumbai EC2 Instance

Switch to the Mumbai AWS Region (`ap-south-1`) and navigate to **EC2** → **Instances**. Click **Launch instances** and configure the instance as follows:

- Name: `Mumbai-EC2`
- AMI: Amazon Linux 2023
- Instance type: `t3.micro`
- Key pair: Select the key pair used for SSH access.

Configure the network settings in the next step before launching the instance.

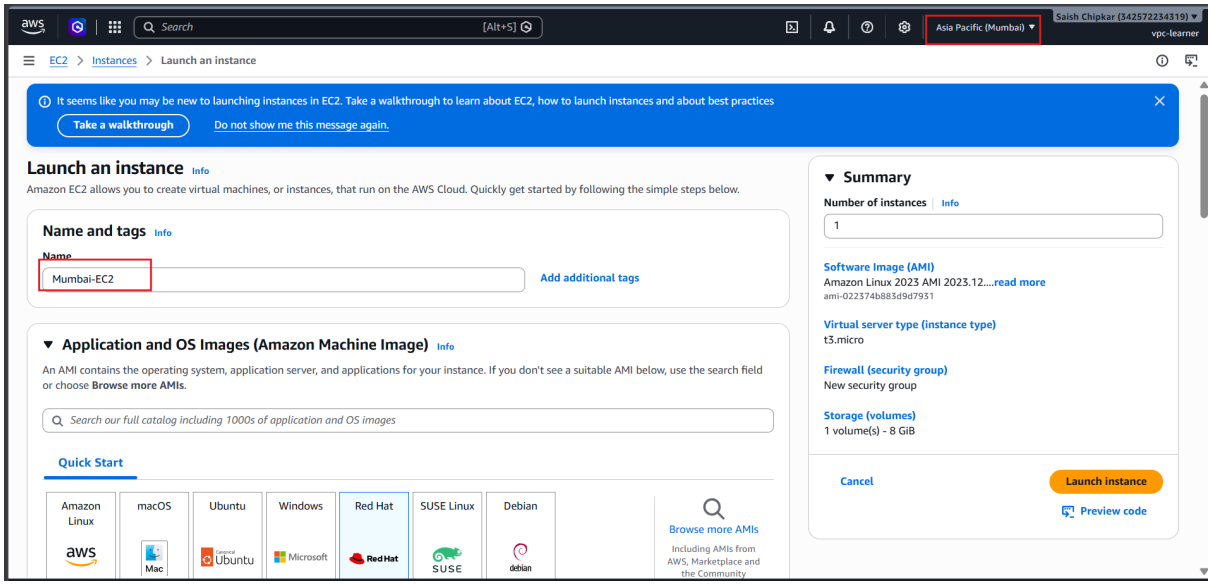


Figure 40: Configuring the Mumbai EC2 instance

5.2 Step 23: Configure the Mumbai Instance Network

Under **Network settings**, configure the following values:

- VPC: `Mumbai-VPC`
- Subnet: `Public-Subnet`
- Auto-assign public IP: Enable
- Security group: Create a new security group.

The selected Mumbai VPC has the CIDR block `10.0.128.0/17`, and the selected public subnet has the CIDR block `10.0.128.0/24`.

Explanation of the network settings

- **VPC:** Places the EC2 instance inside the Mumbai virtual network.
- **Subnet:** Places the instance in the selected subnet within Mumbai-VPC.
- **Auto-assign public IP:** Assigns a public IPv4 address so that the instance can be accessed through the internet when the required routing and security rules are configured.
- **Security group:** Controls inbound and outbound traffic permitted to and from the instance.

For SSH access, allow inbound TCP port 22 only from your own public IP address where possible. To test connectivity using `ping`, also allow inbound ICMP IPv4 traffic from the

Ohio VPC CIDR block, `10.1.0.0/16`. Ensure the outbound rules permit the required traffic.

Click **Launch instance**.

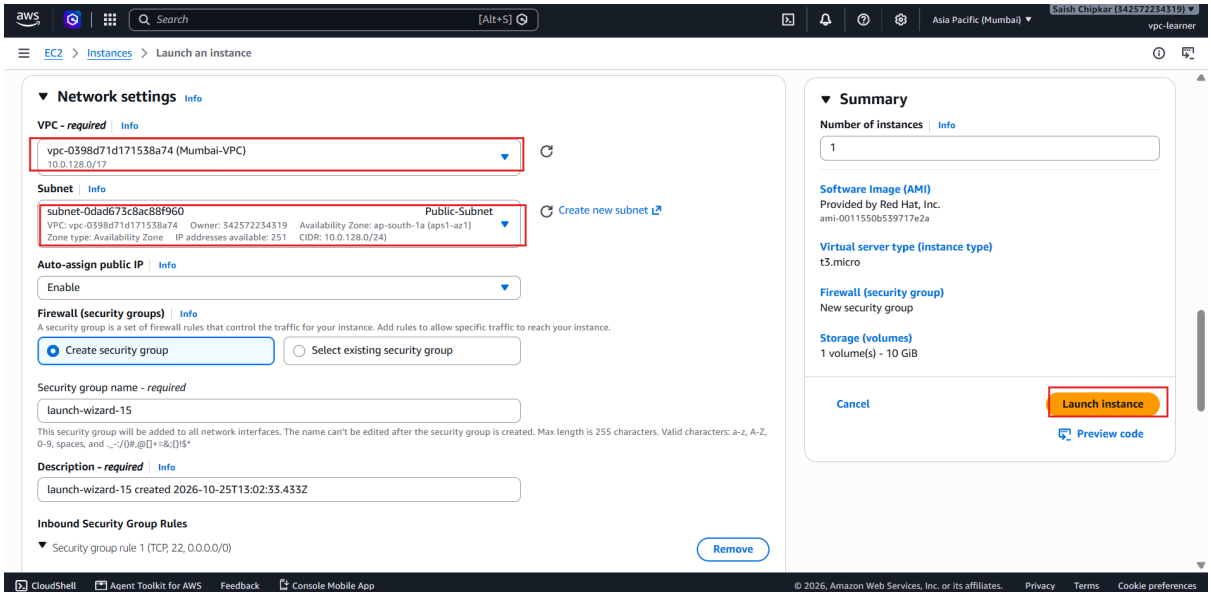


Figure 41: Selecting Mumbai-VPC, its public subnet, and network settings

5.3 Step 24: Launch the Ohio EC2 Instance

Switch the AWS Region to Ohio (`us-east-2`).

Navigate to **EC2** → **Instances** and click **Launch instances**.

Configure the following settings:

- Name: `Ohio-EC2`
- AMI: Amazon Linux 2023
- Instance type: `t3.micro`
- Key pair: Select the key pair used for SSH access.

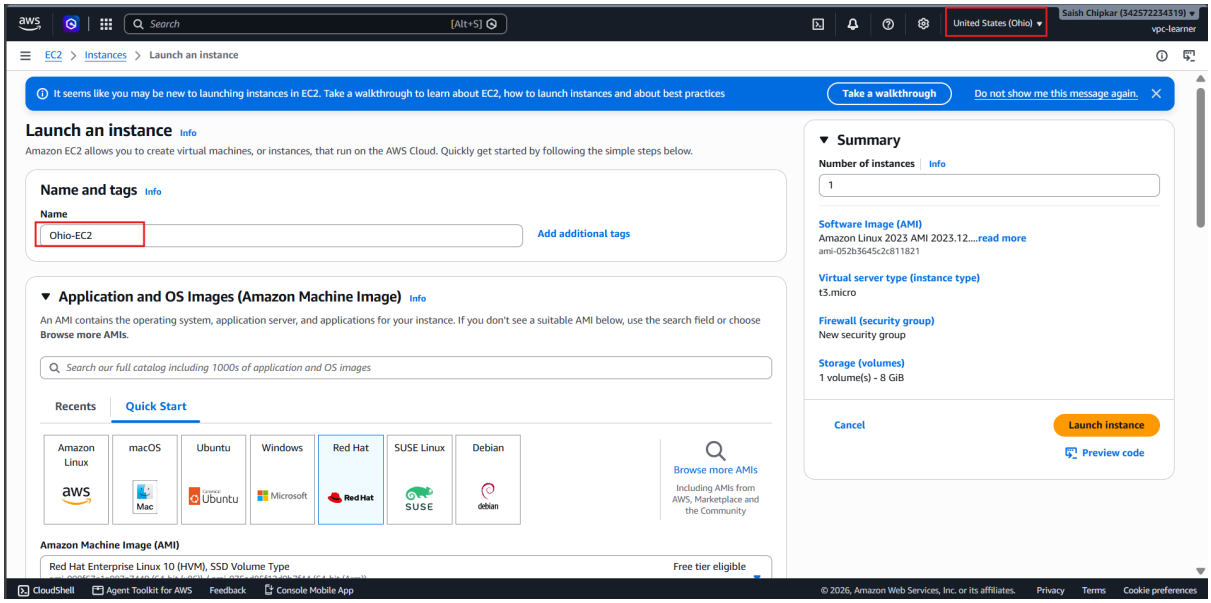


Figure 42: Configuring the Ohio EC2 instance

5.4 Step 25: Configure the Ohio Instance Network

Under **Network settings**, configure the following:

- VPC: `Ohio-VPC`
- Subnet: `Public-Subnet`
- Auto-assign public IP: Enable
- Security group: Create a new security group.

The Ohio VPC uses the CIDR block `10.1.0.0/16`, while its selected public subnet uses `10.1.0.0/24`.

Explanation of the network settings

- **VPC:** Places the EC2 instance inside the Ohio virtual network.
- **Subnet:** Places the instance within the selected subnet of Ohio-VPC.
- **Auto-assign public IP:** Assigns a public IPv4 address for SSH access from outside the VPC, subject to routing and security group rules.
- **Security group:** Controls the inbound and outbound traffic permitted to and from the instance.

Additional ICMP configuration for ping To test connectivity between the Mumbai and Ohio EC2 instances, add an inbound rule to the Ohio instance's security group with the following settings:

- Type: All ICMP - IPv4
- Source: `10.0.128.0/17`

Note: This ICMP rule allows the Mumbai instance to send ping requests to the Ohio instance. It is required for the ping test, but not for SSH access. Configure the rule separately in the security group; it is not shown in the provided screenshots.

Configure the outbound rules to permit the required traffic, then click **Launch instance**.

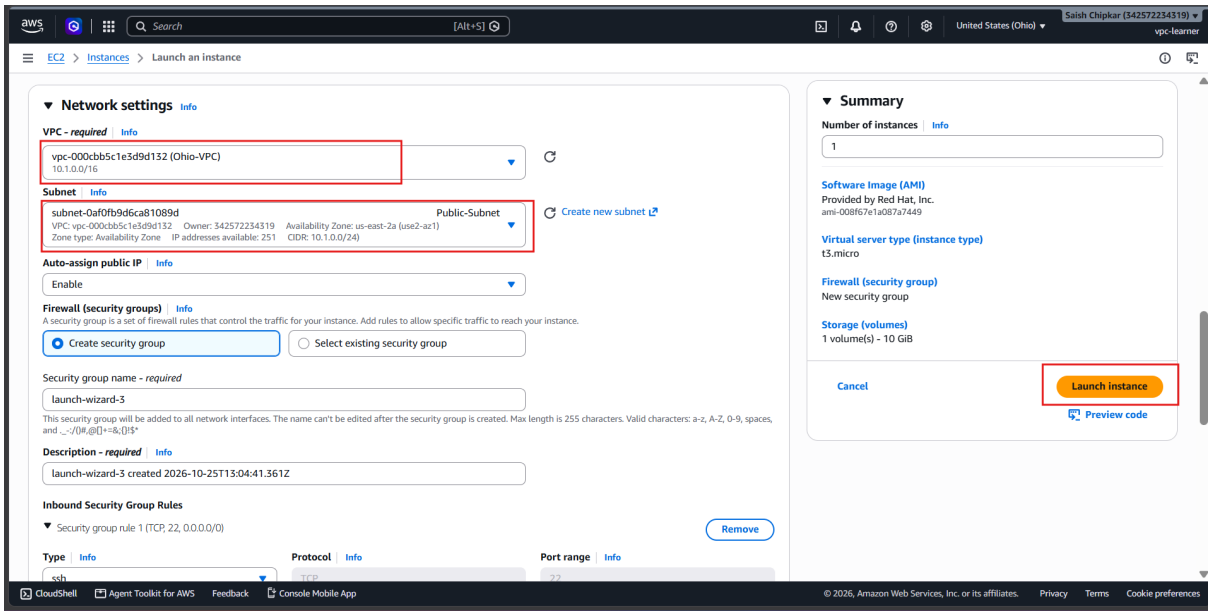


Figure 43: Selecting Ohio-VPC and its public subnet

5.5 Step 26: Connect to Mumbai EC2 and Test Connectivity

After both EC2 instances have been launched, verify that they are running and note their private IP addresses.

In this experiment, the Mumbai instance has the private IP address `10.0.128.178`, and the Ohio instance has the private IP address `10.1.0.184`.

Open Command Prompt on your local computer and navigate to the directory containing the Mumbai instance's private key file.

Connect to the Mumbai EC2 instance using SSH:

```
cd Downloads
ssh -i "key18.pem" ec2-user@<Mumbai-Public-IP>
```

Replace `<Mumbai-Public-IP>` with the actual public IP address of the Mumbai instance.

Once connected, execute the following command:

```
ping 10.1.0.184
```

This command sends ICMP Echo Request packets from the Mumbai instance to the private IP address of the Ohio instance.

```
C:\Users\Saish Chipkar>cd downloads

C:\Users\Saish Chipkar\Downloads>ssh -i "key18.pem" ec2-user@3.6.94.36
The authenticity of host '3.6.94.36 (3.6.94.36)' can't be established.
ED25519 key fingerprint is SHA256:dYmJk5G2XcjR63zASUIGPWLSQp806FiHjjM9WUqbdCc.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '3.6.94.36' (ED25519) to the list of known hosts.
Register this system with Red Hat Lightspeed: rhc connect

Example:
# rhc connect --activation-key <key> --organization <org>

The rhc client and Red Hat Lightspeed will enable analytics and additional
management capabilities on your system.
View your connected systems at https://console.redhat.com/insights

You can learn more about how to register your system
using rhc at https://red.ht/registration
[ec2-user@ip-10-0-128-178 ~]$ ping 10.1.0.184
PING 10.1.0.184 (10.1.0.184) 56(84) bytes of data.
64 bytes from 10.1.0.184: icmp_seq=1 ttl=64 time=203 ms
64 bytes from 10.1.0.184: icmp_seq=2 ttl=64 time=203 ms
64 bytes from 10.1.0.184: icmp_seq=3 ttl=64 time=203 ms
64 bytes from 10.1.0.184: icmp_seq=4 ttl=64 time=203 ms
64 bytes from 10.1.0.184: icmp_seq=5 ttl=64 time=203 ms
64 bytes from 10.1.0.184: icmp_seq=6 ttl=64 time=203 ms
64 bytes from 10.1.0.184: icmp_seq=7 ttl=64 time=203 ms
64 bytes from 10.1.0.184: icmp_seq=8 ttl=64 time=203 ms
64 bytes from 10.1.0.184: icmp_seq=9 ttl=64 time=203 ms
```

Figure 44: Successful SSH connection to Mumbai EC2 and ping test to Ohio EC2

5.6 Step 27: Understand the Network Configuration

The successful ping demonstrates that ICMP packets can travel from the Mumbai EC2 instance to the Ohio EC2 instance and that Echo Reply packets can return.

The communication path is as follows:

1. The Mumbai EC2 instance generates an ICMP Echo Request addressed to `10.1.0.184`.
2. The Mumbai subnet's route table contains a route for `10.1.0.0/16` pointing to the VPC peering connection.
3. The peering connection carries the traffic privately from Mumbai-VPC to Ohio-VPC.
4. The Ohio VPC delivers the packet to the destination EC2 instance through its local route.

5. The Ohio instance sends an ICMP Echo Reply back to the Mumbai instance.
6. The Ohio route table routes the return traffic for `10.0.128.0/17` through the same peering connection, allowing the reply to reach Mumbai.

Important distinction The Internet Gateway and the VPC peering connection serve different purposes. The Internet Gateway provides a route for internet-bound traffic, while the peering connection provides private connectivity between the two VPCs.

Although both instances have public IP addresses for SSH access, the ping test uses the Ohio instance's private IP address. The inter-VPC traffic therefore travels through the peering connection rather than using the public internet.

6 Result

Two EC2 instances were launched in separate VPCs located in different AWS Regions: Mumbai (`ap-south-1`) and Ohio (`us-east-2`).

The VPC peering connection, route tables, subnet associations, and security group rules were configured to permit communication between the instances.

A successful ping from the Mumbai EC2 instance to the Ohio EC2 instance's private IP address (`10.1.0.184`) verified private network connectivity between the two VPCs.